

SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers

¹Tsinghua University, ²ByteDance Seed, ³M-A-P, ⁴TokenWave.AI

Full author list in Contributions

Abstract

Looped Transformers increase effective depth by iterating a shared block of layers, but most evaluations compare at fixed model size, conflating architectural advantage with extra FLOPs. We study looping on Mixture-of-Experts Transformers while closely matching per-token FLOPs, total non-embedding parameters, and KV cache. Through a series of ablations, we arrive at a recipe we call SMELT (**S**parse **MoE** Transformer, middle layers **Lo**op **T**wice), which loops the middle half of layers twice while matching the unlooped Baseline on all three budgets. We scale SMELT across four sizes up to 54B non-embedding parameters and fit a separate Chinchilla-style scaling law for each architecture. SMELT’s loss drops faster with compute, saving 6.8–18.0% of training FLOPs on the compute-optimal frontier. The advantage transfers to downstream benchmarks beyond what validation loss predicts, is largest on Code, and grows with sample length and the number of in-context examples. Mechanistic analysis shows that the second visit reduces the attention sink and redirects mass toward content-relevant tokens, an inductive bias that may underlie the observed performance gains. These results show that looping can improve Transformers even under budget matching, offering a practical recipe that turns depth reuse into measurable gains.

Date: August 10, 2026

1 Introduction

Looped Transformers increase a model’s effective depth by repeating a shared block of layers rather than stacking new ones [18]. The idea has drawn intense recent interest: looped models match or exceed unlooped models several times their size on arithmetic, multi-hop induction, and math [23, 56, 70], and learn algorithmic procedures in context at a fraction of the parameter count [66]. Since effective depth sets how many sequential computation steps each token receives [42, 45], looping promises reasoning capability without additional parameters.

But a fraction of the parameter count is not a fraction of the cost: looping a 12-layer model to 24 executed layers stores half the weights, yet spends roughly a 24-layer model’s per-token FLOPs and needs its full KV cache. Prior evaluations often keep the stored parameter count fixed while increasing recurrent depth [23, 49], or emphasize parameter efficiency relative to larger untied models [66, 70]; in either case the reported gains conflate architectural advantage with uncontrolled extra computation. Whether looping has an architectural advantage beyond this extra computation is unknown. Schwethelm et al. [57] control for this: they hold per-token FLOPs fixed for a dense model and find that r recurrences contribute like $r^{0.46}$ unique-block equivalents, but fixing FLOPs also shrinks the looped model’s unique parameters, so the deficit may reflect parameter loss rather than a flaw in looping itself. A clean answer requires holding three budgets all fixed at once: 1) *per-token FLOPs*, which set training and inference cost; 2) *total parameters*, which bound knowledge

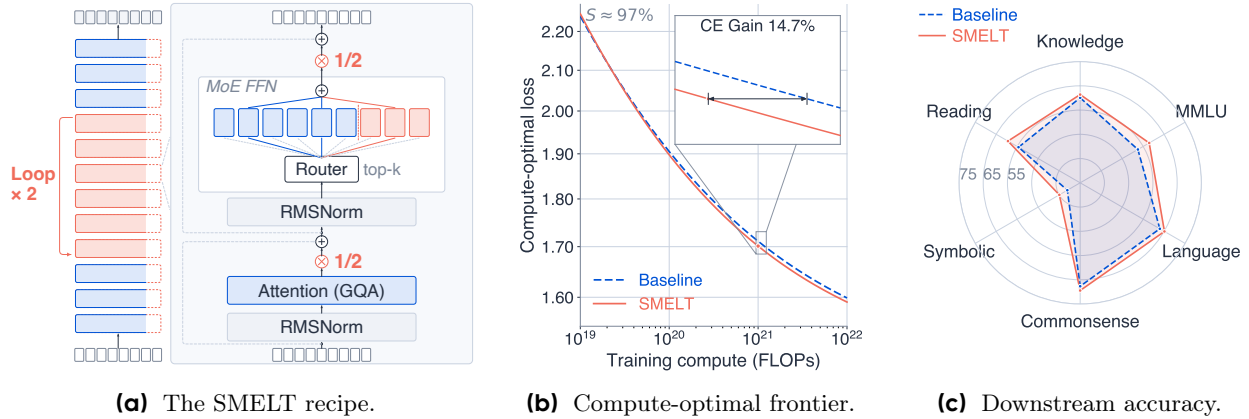


Figure 1 SMELT overview and main results. (a) The SMELT recipe on a compute-matched MoE Baseline: loop the middle 50% of layers twice, narrow the hidden dimension, raise the expert count to recover total parameters, and scale the looped residuals by $1/2$, alongside a smaller attention head size at a higher GQA ratio that holds KV cache nearly unchanged across the extra layer executions. Together they closely match per-token FLOPs, total non-embedding parameters, and KV cache (Section 3). (b) Compute-optimal loss under separate Chinchilla-style scaling fits at compute-equivalent sparsity $S \approx 97\%$ (defined in Section 3.1). SMELT’s frontier drops faster; at $C = 10^{21}$ FLOPs it reaches the same loss with 14.7% less compute (Section 4). (c) Downstream accuracy at the 1.6B scale (54B non-embedding parameters, $S \approx 97\%$), over the five DCLM Core categories (Appendix E), abbreviated on the axes as Reading (Comprehension), Knowledge (World Knowledge), Commonsense (Reasoning), Language (Understanding) and Symbolic (Problem Solving), with MMLU on its own axis. Each axis is the unweighted mean accuracy over its tasks. SMELT leads the Baseline on all six axes.

capacity; and 3) *KV cache*, which limits servable context length.

Mixture-of-Experts Transformers make this budget matching feasible: a looped model can pay for its extra visit by narrowing the hidden dimension, and because MoE decouples total parameters from per-token FLOPs [20, 58], it can recover the lost capacity by raising expert count rather than giving it up. KV cache parity is restored separately by adjusting the head size and GQA ratio. Under this matching, we search the loop design space through three ablations and find three rules: 1) *loop the middle half* of layers rather than the full stack, 2) *give the looped model a larger effective depth-to-width ratio* than the Baseline, and 3) *loop twice* rather than three or four times. Together these lock in a recipe we call **SMELT**, a **S**parse **M**oE Transformer whose middle layers **Loop Twice** (Figure 1; Section 3).

We scale the recipe up to 54B non-embedding parameters across four model sizes and four sparsity levels, comparing SMELT against the Baseline at each grid point. SMELT reaches lower loss than the Baseline at every scale and sparsity level. To predict how the advantage evolves at scale, we fit separate Chinchilla-style scaling laws [28] for the two architectures. SMELT’s loss drops faster with both compute and data, and the CE Gain increases across the fitted compute range (Section 4).

Beyond aggregate validation loss, we examine downstream benchmarks and per-domain performance. On DCLM [41] and MMLU [25], SMELT outperforms the Baseline in nearly all matched pairs, with an improvement that exceeds what the validation loss gap alone would predict. We also find that among training domains, Code benefits the most. The advantage is further amplified on longer samples and when more in-context demonstrations are available, suggesting that looping becomes more valuable as the input grows richer in retrievable structure (Section 5).

To build intuition for the advantages observed above, we examine what the looped layers compute differently on their second visit. We find that the experts selected and the tokens attended to overlap substantially across the two visits, while the residual updates grow substantially larger and stay aligned, indicating that the second visit amplifies the first rather than overwriting it. Within the attention mechanism, values change more than queries and keys, and the attention sink is broadly reduced, redirecting mass toward content-relevant tokens,

which may relate to the observed ICL advantage (Section 6).

Our results show that looping can improve Transformers even under budget matching, with particularly strong gains on structured data and in-context learning tasks. Together with the growing body of work exploring looped language models from complementary angles [23, 49, 56, 57, 70], they suggest that looping is a robust and increasingly well-understood design axis.

2 Related Work

Looped Transformers. The Universal Transformer [18] introduced weight sharing across depth, applying a shared block iteratively to decouple effective depth from unique parameter count. Huggins [23] scales this idea to 3.5B parameters with a prelude–recur–coda layout (2–4–2 layers at its largest scale) that loops the middle 50%, establishing the template that most subsequent work adopts: an unshared entry block, a shared middle block that is iterated, and an unshared exit block. Ouro [70] takes a different partition: it loops the entire L -layer stack at 1.4–2.6B. Both show that looped models match or exceed much larger unlooped models on reasoning benchmarks. A recurring finding across this body of work is that the middle layers benefit most from repetition. Saunshi et al. [56] compare looped models against both iso-parameter and iso-FLOP baselines: looped models have worse perplexity than the iso-FLOP baseline but disproportionately improve reasoning tasks, and iterating only a middle span, the prelude–recur–coda layout above, further improves perplexity over full-stack looping. Kapl et al. [32] report that, among the block placements tested in their setting, adapting a localized middle block for looping yields the strongest overall results. A related but untied construction appears in SOLAR [34], whose depth-up-scaling method also targets the middle layers, duplicating them to grow a pretrained model before continued pretraining. ALBERT [37] shares weights across all layers for parameter efficiency, an early instance of full cross-layer tying that predates the partial-loop designs above. Beyond the choice of which layers to loop, other work explores how to loop: Relaxed Recursive Transformers [4] add per-layer LoRA adapters to relax strict weight tying, Mixture-of-Recursions [5] routes each token to a learned recursion depth, and Think-at-Hard [21] learns when to iterate on hard tokens. Among these studies, Saunshi et al. additionally report an iso-FLOP comparison in which the looped model uses fewer unique parameters than the untied baseline [56]. RINS [3] matches total training compute by training its early-block recursive model on fewer tokens than its baseline, whereas we additionally match the token budget. Most other works above compare looped models against parameter-matched baselines, so the reported gains conflate architectural advantage with the extra FLOPs that repetition spends; whether looping helps or hurts under jointly matched compute and token budgets remains open.

Mixture-of-Experts and loop recurrence. Answering the question above requires holding three budgets fixed at once: per-token FLOPs, total parameters, and KV cache (Table 1). MoE makes this feasible by decoupling total parameters from per-token FLOPs [20, 58]: a looped model can pay for its second visit by narrowing the hidden dimension, then recover the lost capacity by adding experts. MoEUT [15] proposes an MoE-based Universal Transformer that, for the first time, matches standard Transformers on language modeling. Lee et al. [38] find that sparse layers are critical to scaling looped models, because routing divergence across visits recovers expressivity lost to weight tying. Jaggi [30] similarly reuses experts across middle layers and reinvests the saved parameters in larger expert pools. Several concurrent works train MoE Looped Transformers at scale: LoopMoE [10] matches total parameters and per-token FLOPs at 3B and 9B; Gao et al. [22] train 6B and 20B models under wall-clock matching and independently find two loop steps optimal, corroborating our loop-count ablation (Section 3.5). Knupp et al. [35] (DREAMER) recurrently apply a single MoE layer 16–32 times with attention over previous depths under matched FLOPs, parameters, and memory; we view this single-layer recurrence as closer to implicit-depth models than to the block-level looping studied here. Chen et al. [11] share a universal expert pool across layers, converting depth into virtual MoE width rather than looping the full layer block. These concurrent studies show that looping improves MoE models at individual scales. We ask a complementary question: does the advantage hold across a full scaling ladder when per-token FLOPs, total parameters, and KV cache are all matched? For block-level looping, our comparison is the first multi-scale study that closes all three budgets simultaneously and fits a separate scaling surface for each architecture, converting point observations into an attributable and extrapolable claim (Section 4).

Table 1 Comparison of looped Transformer studies in terms of architecture, loop span, matched budgets, scaling analysis, and key finding. $\checkmark$ = matched; $\times$ = unmatched or grows with loop count; $(\checkmark)$ = partially matched (see notes). Span denotes the fraction of the layer stack that is repeated, and r denotes the number of times that span executes per token (both are defined in Section 3.1).

Work	Arch.	Span	Matched budgets			Scaling analysis	Key finding
			FLOPs/token	Stored params	KV cache		
Huginn [23]	Dense	Mid. 50%	$\times$	$\checkmark$	$\times$	Fixed size (3.5B)	Prelude-recur-coda looping improves reasoning; extra FLOPs uncontrolled
Ouro [70]	Dense	Full (100%)	$\times$	$\checkmark$	$\times$	Fixed size (1.4–2.6B)	Full-stack looping improves reasoning; extra FLOPs uncontrolled
Saunshi et al. [56]	Dense	Mid. 33%	$(\checkmark)^a$	$\times$	$\times$	Fixed size (170M–1B)	Middle-block looping lowers perplexity; inductive bias toward reasoning
Schwethelm et al. [57]	Dense	Mid. ^c	$\checkmark$	$\times$	$\checkmark$	Iso-depth law (25M–1.6B)	r recurrences $\approx r^{0.46}$ unique equivalents; looping loses
Prairie et al. [49]	Dense	Mid. 33%	$\times$	$\checkmark$	$\times$	Scaling law at fixed params (100M–1.3B)	Optimal r grows with compute; FLOPs and KV grow with r
Lee et al. [38]	MoE	Full (100%)	$\checkmark$	$\times$	$\checkmark$	IsoFLOP law (16M–305M active)	Sparse routing mitigates the loop penalty; Looped-MoE beats dense Base but trails MoE in loss
LoopMoE [10]	MoE	Mid. 33%	$\checkmark$	$\checkmark$	$\times$	Fixed size (3B, 9B)	Wins 8/9 benchmarks at 3B and 9/9 at 9B
Gao et al. [22]	MoE	Each layer	$(\checkmark)^b$	$\times$	$\times$	Fixed size (6B, 20B)	Wins; $r=2$ optimal; memory savings reinvested into width
SMELT (ours)	MoE	Mid. 50%	$\checkmark$	$\checkmark$	$\checkmark$	Scaling ladder across 0.1B–54B non-embedding parameters	Wins; 6.8–18.0% FLOPs saving on validation loss; benchmark gains exceed validation loss prediction

^a Matched-FLOPs perplexity comparison for the prefix-loop-suffix layout; the accuracy-vs-loop-count results are at fixed model size. ^b Matches wall-clock training time rather than per-token FLOPs. ^c Physical span varies with loop count (approx. 67%, 50%, 33% for $r=2, 3, 4$).

Scaling laws. Kaplan et al. [33] established power-law relations between loss, model size, and training tokens. Hoffmann et al. [28] (Chinchilla) refined the compute-optimal allocation and showed that parameters and tokens should scale at roughly equal rates with compute. For MoE models, Clark et al. [12] and Abnar et al. [1] extend these laws with routing variables such as expert count and sparsity; Li et al. [40] study the conditions under which MoE surpasses dense models at equal resources. Recent work extends these analyses to looped architectures. Schwethelm et al. [57] fit a joint scaling law under an iso-depth protocol that holds effective depth (and therefore FLOPs and KV cache) fixed while reducing unique parameters. In their law, the effective parameter count of the recurrent block is $N_{\text{once}} + r^{0.46} N_{\text{rec}}$: r recurrent executions contribute like $r^{0.46}$ unique-block equivalents. The sublinear exponent means recurrence yields diminishing capacity per iteration, so the looped model loses at matched compute. Prairie et al. [49] fit compute-optimal recurrence at fixed unique parameter count, but depth, inference FLOPs, and KV cache all grow with r , so the fit cannot separate the per-parameter sharing cost from the growing budgets. These two results bracket the question from opposite sides: Schwethelm et al. price looping’s cost at fixed depth, Prairie et al. find looping’s benefit at fixed parameters. We fit a separate Chinchilla-style surface for each architecture on MoE models while closely matching per-token FLOPs, total non-embedding parameters, and KV cache, which lets us isolate the architectural effect and measure the compute saving on the frontier (Section 4).

Attention sinks. Autoregressive Transformers concentrate attention mass on initial tokens regardless of semantic content. Xiao et al. [65] identify this attention-sink phenomenon and exploit it for streaming inference; Gu et al. [24] characterize when sinks emerge during pretraining; Sun et al. [60] link them to massive activations in the residual stream, with an analogous register effect in vision models [16]; Barbero

et al. [6] argue that sinks counteract over-mixing and therefore strengthen as depth grows, a trend also observed geometrically by Ruscio et al. [54]; and Qiu et al. [50] show that a query-dependent sigmoid gate after attention strongly mitigates the sink in the evaluated models. These works treat the sink as a static property of a trained model. Our mechanistic analysis (Section 6) adds a dynamic observation: within a single forward pass, a second weight-tied visit reduces sink mass and redirects it toward content-relevant tokens.

3 Design Recipe under Matched Compute

3.1 Training protocol

We write **Baseline** for the standard unlooped MoE Transformer and **Looped Transformer** for the variant that repeats a contiguous span of m layers r times. The ablations in Sections 3.3–3.5 sweep both m and r ; the values they select define SMELT in Section 3.5. All runs share the same architecture template, optimizer, data, and evaluation. Only the loop configuration (span, count, and effective depth) and the matched width/expert/head hyperparameters (Section 3.2) differ between the two. In the Looped Transformer, each sublayer’s residual update within the looped span is scaled by $1/r$ where r is the loop count [64] (Figure 1a); this prevents correlated weight-tied updates from inflating the residual stream across visits.

Architecture. Each model is a decoder-only Transformer with sparse MoE feed-forward layers. Every MoE layer routes each token to its top-8 experts. Attention uses grouped-query attention (GQA) [2]. We adopt an internal Baseline family and use its active non-embedding parameter count to label four matched scales: 100M, 200M, 600M, and 1.6B (physical depth $L = 10, 12, 20, 30$). Appendix A lists the configuration of every Baseline in the grid.

Compute-equivalent sparsity. An MoE model stores N parameters but activates only a fraction per token. We measure this fraction against a fully active control: an unlooped Baseline that activates all experts per token at the same scale, so its per-token FLOPs reflect 100% utilization of total parameters. Let F_0 and N_0 be this control’s per-token training FLOPs and total non-embedding parameters. For any configuration with per-token FLOPs F and N total non-embedding parameters, we define

$$N_{\text{act}}^{\text{eq}} = \frac{F}{F_0} N_0, \quad S = 1 - \frac{N_{\text{act}}^{\text{eq}}}{N}. \quad (1)$$

$N_{\text{act}}^{\text{eq}}$ scales the control’s parameter count by the FLOPs ratio: a model that spends half the control’s FLOPs is treated as activating half the parameters. S is the inactive share, analogous to the parameter-ratio sparsity of Abnar et al. [1] but computed from FLOPs ratios to account for context-dependent attention cost; the control maps to $S = 0$ by construction. A Looped Transformer matched to this control at $S = 0$ is not dense. Matching (Section 3.2) narrows H and raises expert count to recover total parameters, so the Looped Transformer routes to a top-8 subset of a larger pool even at $S = 0$ (e.g., 16 experts per layer at 200M). $S = 0$ means the per-parameter compute intensity matches the fully active Baseline, not that every expert is activated. We report matched Baseline / Looped Transformer pairs at $S \approx 85\%$, 95% , and 97% ; the same label applies to both architectures in all figures.

Optimizer and schedule. We use AdamW [43] with a warmup-stable-decay (WSD) schedule [29]. The stable phase trains at a constant learning rate over 196,075 steps with a global batch size of 256 sequences ($\approx 1\text{M}$ tokens per step). From each stable run we branch six cosine-decay schedules at steps 10,000, 20,000, 50,000, 100,000, 150,000, and 196,075, each decaying over 10B additional tokens. The six branch points give six token horizons per configuration.

Data. We pretrain on an internal corpus. The stable phase of the WSD schedule consumes 205B tokens; each cosine-decay branch then trains on 10B additional tokens drawn from fresh data not seen during the stable phase, bringing the longest branch to approximately 215B tokens with no repeated data. All matched Baseline / SMELT pairs at the same scale and sparsity level are trained on identical token sequences. The held-out validation set covers 39 individual sources grouped into five categories: Code, Math/STEM, Knowledge,

Finance, and Web. Training sequences are packed into 4096-token contexts with segment-level attention masks, where each segment is one document within the packed context.

Evaluation. We score three metrics throughout. Validation loss is the token-weighted cross-entropy on the held-out validation set, averaged across all 39 sources. The other two come from the 22-task DCLM Core suite [41], evaluated with 10 random few-shot seeds per few-shot task and BOS-preserving truncation (details in Appendix E). The seven zero-shot tasks in the suite are deterministic and run once. DCLM Core is the mean centered accuracy over the 22 tasks (rescaled so that the DCLM-v2 reference baseline maps to zero). We define *DCLM Completion* as the token-weighted cross-entropy on the gold answer tokens of the nine tasks in the suite that ask for a free-form answer rather than a choice among options (lower is better). We also run MMLU 5-shot [25] and report it on its own, as it is widely used as a standalone benchmark. To prevent benchmark contamination, we filtered known evaluation items from the pretraining corpus before training.

3.2 Matching as a compute-allocation problem

We care about three quantities when we compare the Looped Transformer with the Baseline. Total parameters control how much knowledge a model can memorize; per-token FLOPs set inference and training cost; KV cache determines the longest context a deployment can serve. To attribute performance gaps primarily to architecture, we closely match all three between the Looped Transformer and the Baseline.

Consider the cost of looping. A standard MoE Transformer with L physical layers spends per-token training FLOPs proportional to $L \cdot H^2$, where H is the hidden dimension. When we loop a contiguous span of m layers r times, the effective depth grows to $L_{\text{eff}} = L + (r-1)m$, and FLOPs grow in proportion, while total parameters stay the same. To keep per-token FLOPs fixed, we must shrink the model elsewhere: we narrow H or reduce L . That shrinkage also cuts total parameters and KV cache, so we must compensate on three axes at once.

Table 2 lists the three budgets, the adjustment that holds each, and the residual mismatch after matching. Suppose we want a Looped Transformer at 200M, $S \approx 95\%$ that loops the middle 6 of its 12 layers twice, executing 18 layers in total. The matched Baseline has $L = 12$, hidden dimension $H = 1280$, and 192 experts per layer (1.33×10^9 training FLOPs per token, 3.87×10^9 total non-embedding parameters). The 6 extra layer executions cost FLOPs, so we narrow H from 1280 to 1056 to keep per-token FLOPs close. Narrowing H also shrinks every expert’s FFN, which reduces total parameters. To recover them, we raise the per-layer expert count from 192 to 288. Expert count is uniform across all layers in both architectures. After both adjustments, per-token FLOPs land at 1.37×10^9 (+2.9% vs. the Baseline’s 1.33×10^9), total parameters at 3.89×10^9 (+0.4%), and KV cache within 4%. Both FLOPs counts are measured at the mean document length within the packed 4096-token contexts (since segment-level masks restrict attention to each document) and include the attention cost that scales with that length.

Table 2 Budget-matching adjustments and typical residual mismatch. Exact per-configuration ratios are reported in Appendix A.

Budget	Adjustment	Typical mismatch
Per-token FLOPs	Shrink H , or change L	< 4%
Total parameters	Add experts	< 1%
KV cache	Head size or GQA ratio	< 4%

We apply this matching at each compute-equivalent sparsity level $S \approx 85\%$, 95% , and 97% (Section 3.1). With matching in place, we search the Looped Transformer design space through three ablations at 200M: which layers to loop, how deep the model should be, and how many passes to run.

3.3 Looping the middle half beats full looping

We first ask which layers should loop, starting with two passes. This ablation fixes physical depth at $L = 12$ and varies which contiguous middle segment loops twice. The segment length ranges from 0 (Baseline) through 12 (full looping). We run the sweep at $S \approx 85\%$ and $S \approx 95\%$ to test whether the peak moves with sparsity.

Table 3 Loop-span sweep at 200M and $L = 12$. Validation loss is minimized near 50% span for both sparsity levels; we adopt this as the default loop span. Bold marks the best value within each sparsity level.

Looped layers	Span (%)	Effective layers	Val. loss ↓	DCLM Core ↑	DCLM Completion ↓
$S \approx 85\%$					
0	0	12	1.9445	24.92 ± 0.13	2.2887
2	17	14	1.9384	26.20 ± 0.14	2.2824
4	33	16	1.9275	25.13 ± 0.16	2.2733
6	50	18	1.9257	27.57 ± 0.13	2.2635
8	67	20	1.9374	25.84 ± 0.10	2.2843
10	83	22	1.9413	24.45 ± 0.17	2.2835
12	100	24	1.9322	25.31 ± 0.17	2.2759
$S \approx 95\%$					
0	0	12	1.8735	29.34 ± 0.10	2.2189
2	17	14	1.8562	30.84 ± 0.20	2.2022
4	33	16	1.8524	31.22 ± 0.18	2.1970
6	50	18	1.8517	29.68 ± 0.13	2.1917
8	67	20	1.8544	31.78 ± 0.18	2.1891
10	83	22	1.8572	31.49 ± 0.15	2.1964
12	100	24	1.8601	30.82 ± 0.14	2.2038

Validation loss reaches its minimum near 50% span for both sparsity levels (Table 3). We select the span on validation loss because the DCLM metrics do not track it in this sweep. At $S \approx 95\%$, for instance, DCLM Core peaks at 67% span and dips at 50% span—the opposite of the validation-loss ranking—while validation loss, which averages over billions of tokens, varies smoothly. The reported standard errors (± 0.1 – 0.2 points) reflect variability across the 10 evaluation seeds; they do not capture training-run variability, since each configuration is trained once. We therefore fix the span at 50% for all subsequent experiments and treat the DCLM columns as a consistency check rather than a selection criterion. The optimum agrees with depth-dependent specialization in Transformer representations [36, 62] and with partial-looping studies [9, 32, 56]: looping a middle block consistently outperforms full-stack looping, as the first and last layers serve specialized roles that benefit from independent parameters.

3.4 The Looped Transformer prefers a larger effective depth-to-width ratio

With the span fixed at 50% and still under two passes (Section 3.3), we sweep the depth-to-width ratio for both the Baseline and the Looped Transformer at 200M, $S \approx 85\%$ (Table 4). At each depth, we re-match the remaining architectural hyperparameters under the FLOPs, parameter, and KV-cache constraints of Section 3.2.

Table 4 Effective-depth sweep at 200M, $S \approx 85\%$, 50% loop span. Physical depth is unique layers; effective depth is layers run in a forward pass. The 12/18 Looped Transformer wins on all three metrics; its physical depth matches the Baseline’s validation loss optimum (12). Bold marks the best value within each architecture.

Physical depth	Effective depth	Val. loss ↓	DCLM Core ↑	DCLM Completion ↓
Baseline				
9	9	1.9638	24.01 ± 0.14	2.2983
12	12	1.9445	24.92 ± 0.13	2.2887
15	15	1.9457	25.84 ± 0.15	2.2824
18	18	1.9542	24.26 ± 0.14	2.2967
Looped Transformer				
8	12	1.9591	24.39 ± 0.17	2.2932
10	15	1.9389	25.93 ± 0.14	2.2871
12	18	1.9257	27.57 ± 0.13	2.2635
14	21	1.9433	26.55 ± 0.23	2.2766

The Looped Transformer’s optimal effective depth-to-width ratio is larger than the Baseline’s (Table 4): the

Baseline peaks at physical depth 12, while the Looped Transformer peaks at 12/18 (physical 12, executed 18). On validation loss, the winning Looped Transformer has the same physical depth as the Baseline (12), so we set the Looped Transformer’s physical depth equal to the Baseline’s in all subsequent experiments.

We hypothesize that the Looped Transformer can sustain a larger effective depth-to-width ratio because its additional execution depth does not increase its physical depth: the shared layers receive gradient contributions from multiple visits, including later occurrences with shorter paths to the output. This multi-depth gradient signal may make additional serial computation easier to optimize than an equally deep stack of independently parameterized layers.

3.5 Two loops beat three or four

Sections 3.3–3.4 fix the span at 50% and the physical depth at the Baseline’s optimum. The remaining question is how many times to loop those middle layers. Table 5 reports the 200M, $S \approx 85\%$ sweep from one to four visits. Extra visits cost FLOPs, so the budget-matching procedure of Section 3.2 narrows width to stay on the same budget line; more loops therefore mean a thinner model.

Table 5 Loop-count sweep at 200M, $S \approx 85\%$, 50% loop span, physical depth matched to the Baseline. Two visits give the lowest validation loss and highest DCLM Core; more loops force a thinner model under matched FLOPs. Bold marks the best value.

Model	Effective depth	Val. loss ↓	DCLM Core ↑	DCLM Completion ↓
Baseline (1×)	12	1.9445	24.92 ± 0.13	2.2887
Looped Transformer 2×	18	1.9257	27.57 ± 0.13	2.2635
Looped Transformer 3×	24	1.9385	27.15 ± 0.20	2.2779
Looped Transformer 4×	30	1.9360	27.30 ± 0.19	2.2820

Two visits win on all three metrics (Table 5); both a third and a fourth visit regress because the matched FLOPs cap forces a thinner model. The regression is not ordered in the loop count, so only the shared gap to 2× is meaningful.

We therefore fix loop count at 2 for all subsequent scale-up experiments. The three ablations together lock the recipe we call **SMELT**, a **S**parse **M**oE Transformer whose middle layers **Loop Twice**, through three rules: 1) *loop the middle half* of layers rather than the full stack (Section 3.3), 2) *give the looped model a larger effective depth-to-width ratio* than the Baseline (Section 3.4), and 3) *loop twice* rather than three or four times.

4 Scaling Laws and Compute Savings

The three ablations in Section 3 locked the SMELT recipe, but they all ran at the 200M scale (up to 3.9B non-embedding parameters). Does the advantage survive when models grow to tens of billions of parameters? This section scales the recipe across four sizes up to 54B non-embedding parameters, fits a separate scaling surface for each architecture, and quantifies how much compute SMELT saves.

4.1 The SMELT recipe scales across the grid

We apply the budget matching protocol and the locked recipe from Sections 3.1–3.5 to the full 4×4 grid: four scales (100M / 200M / 600M / 1.6B) crossed with an $S=0\%$ dense-reference control and three sparse levels $S \approx \{85\%, 95\%, 97\%\}$. Each cell trains a matched Baseline / SMELT pair under the shared WSD schedule from Section 3.1. The six cosine-decay branches per run (Section 3.1) yield six token horizons, which provide the variation along the data axis needed to fit the data term of the scaling law (Section 4.2). Forking decay branches from one stable run is cheaper than training a separate run per horizon, and annealed checkpoints are the ones that reflect converged loss [29, 63]. That gives 32 runs, $16 \times 6 = 96$ matched Baseline / SMELT pairs, and 192 evaluation endpoints.

Figure 2 shows two cells at $S \approx 97\%$: the 600M scale (22B non-embedding parameters) and the 1.6B scale (54B non-embedding parameters). In both cells, the SMELT curve tracks below the Baseline through most of training, and the gap persists into every cosine-decay branch. Appendix B gives the curves for all 16 grid cells.

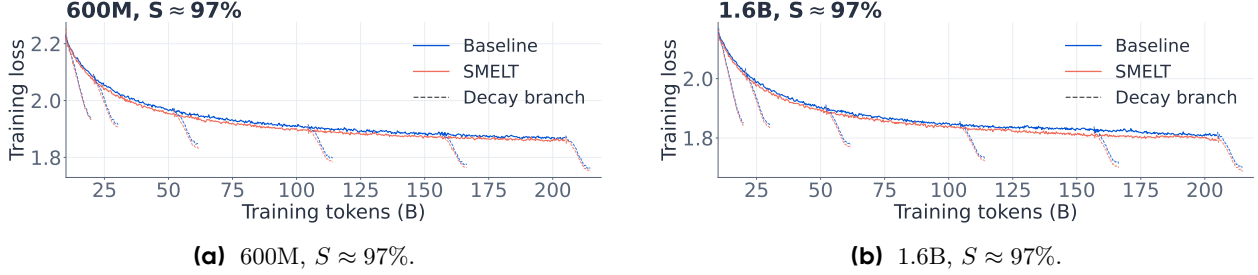


Figure 2 Training loss at $S \approx 97\%$ for two scales. Solid lines are the stable (constant-LR) phase; dashed lines are cosine-decay branches forked at six checkpoints. SMELT (red) runs below the Baseline (blue) throughout.

Figure 3 replaces the token axis with cumulative training FLOPs, a hardware-independent proxy for arithmetic training cost. Each panel is one sparsity level; colors mark the four scales. Open circles mark Baseline endpoints; filled squares mark SMELT.

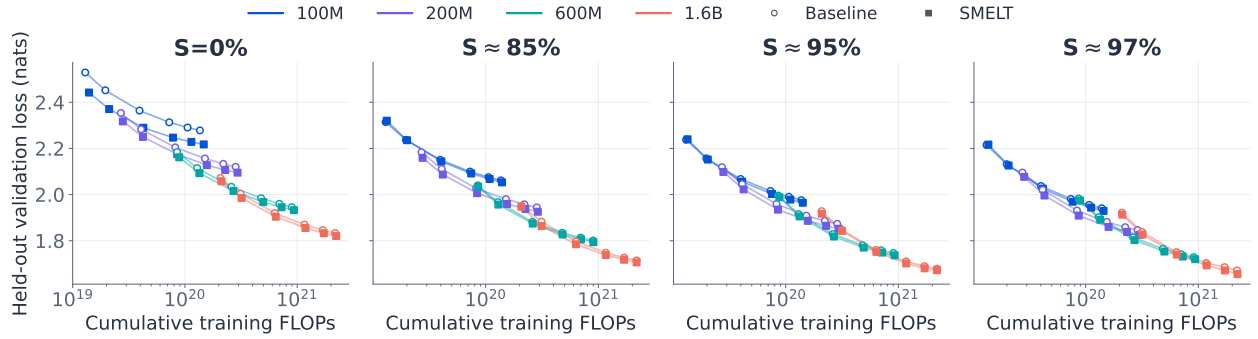


Figure 3 Validation loss vs. cumulative training FLOPs, faceted by compute-equivalent sparsity S and colored by scale. Open circles = Baseline; filled squares = SMELT. At comparable measured compute, SMELT reaches lower loss in every cell.

4.2 Scaling form and compute efficiency gain

Scaling form. Kaplan et al. [33] established power-law relations between loss, model size, and training tokens. The Chinchilla law [28] recasts the joint dependence as an additive form $\mathcal{L}(N, D) = E + AN^{-a} + KD^{-c}$: an irreducible term, a capacity term, and a data term. For MoE Transformers, total parameters N do not determine how many parameters a token effectively activates, so one N axis cannot describe a dense and a sparse model at once. Prior MoE scaling laws therefore extend the capacity term with a variable that describes routing: expert count [12], expert granularity [44], or sparsity [1]. We follow that line: we replace N with per-token FLOPs F and scale the capacity term by compute-equivalent sparsity S from Eq. 1:

$$\mathcal{L}(F, S, D) = E + \frac{A(1-S)^b}{F^a} + \frac{K}{D^c}. \quad (2)$$

F is the measured training FLOPs per token at the mean document length within the packed 4096-token contexts (since segment-level masks restrict attention to each document); it includes the attention cost that scales with that length and therefore exceeds the parameter-only $6N$ approximation. E is the irreducible loss. The middle term is capacity-limited: more per-token FLOPs F shrink it, and so does higher sparsity (lower $1-S$), because a sparser model stores more total parameters for the same per-token cost. The last term is data-limited: more training tokens D reduce it. We fit this form separately for the Baseline and for SMELT, giving each its own six coefficients (E, A, K, a, b, c). There is no shared loop indicator: all six coefficients may differ.

Compute-optimal frontier. Given a total training compute budget C and a sparsity level S , we want to split C between per-token FLOPs F and training tokens $D = C/F$ to minimize loss. Following Chinchilla Approach 3 [28], we minimize $\mathcal{L}(F, S, D)$ subject to $C = F \cdot D$. The capacity term and the data term pull in opposite directions: a larger F shrinks the first but leaves fewer tokens $D = C/F$ for the second. Balancing the two gives the compute-optimal per-token FLOPs

$$F^* = \left(\frac{a A (1-S)^b C^c}{c K} \right)^{1/(a+c)}, \quad (3)$$

and the corresponding optimal tokens $D^* = C/F^*$. By construction, both F^* and D^* are power laws in C , as in Chinchilla. Substituting back, the lowest achievable loss at budget C is

$$\mathcal{L}^*(C, S) = E + \underbrace{f(A, K, a, b, c, S)}_{\text{prefactor}} C^{-\gamma}, \quad \gamma = \frac{a c}{a + c}. \quad (4)$$

The prefactor depends on A , K , a , b , c , and S , while γ controls how quickly the reducible term decreases with compute.

Compute efficiency gain. When two architectures are fitted with separate surfaces, they trace different frontiers. Holding performance fixed and asking how much compute an improvement saves is the standard lens on algorithmic progress [26]. We define the *compute efficiency gain* (CE Gain) of a target architecture over a reference architecture as the fraction of compute the target saves at a common loss:

1. Evaluate the target’s frontier loss at budget C_{tgt} .
2. Invert the reference’s frontier to find the budget C_{ref} at which it reaches the same loss.
3. Compute the fraction saved:

$$\text{CE Gain} = 1 - \frac{C_{\text{tgt}}}{C_{\text{ref}}}. \quad (5)$$

A positive value means the target reaches the same loss with less compute than the reference. Work on algorithmic progress measures the same quantity as a multiplier $C_{\text{ref}}/C_{\text{tgt}}$ [17, 27]. In our case the target is SMELT and the reference is the Baseline.

4.3 SMELT saves training compute on the frontier

Fit protocol. We fit Eq. 2 separately for the Baseline and for SMELT on the sparse grid $S \approx \{85\%, 95\%, 97\%\}$, using the same Huber-loss ($\delta = 10^{-3}$) minimization on log-loss with L-BFGS-B for both architectures [28]. Keeping the procedure identical for both is important, because compute-optimal conclusions are sensitive to fitting choices [48]. Not all endpoints enter the same fit. The $S = 0$ dense-reference control sits far from the three sparse levels; including it in a joint fit worsens the surface RMSE on the sparse grid, because the optimizer compromises between two regimes. We therefore exclude $S = 0$ from the scaling-law fit. The sparse fit uses 72 Baseline endpoints and 72 SMELT endpoints from the three levels $S \approx \{85\%, 95\%, 97\%\}$. These 144 endpoints span 1.3×10^{19} to 2.2×10^{21} cumulative training FLOPs, which is the window the fitted surfaces are supported on.

Fitted coefficients. Table 6 reports the six coefficients for each architecture.

Table 6 Separate-surface coefficients on $S \approx \{85\%, 95\%, 97\%\}$, rounded to four decimal places (A and K to four significant figures in scientific notation). RMSE is in nats on validation loss itself, not on log-loss; the fit objective is Huber loss on log-loss.

Arch	E	A	K	a	b	c	RMSE
Baseline	1.4439	1.366×10^3	1.975×10^6	0.3703	0.1530	0.6594	0.00554
SMELT	1.4493	1.963×10^3	5.264×10^6	0.3892	0.1460	0.7011	0.00952

The exponents a and c decide how fast loss drops with scale and data. SMELT’s capacity exponent a is 0.3892 vs. the Baseline’s 0.3703, and its data exponent c is 0.7011 vs. 0.6594. Both are larger, so SMELT’s reducible loss $\mathcal{L} - E$ drops faster along the compute axis. Combining a and c into the frontier exponent $\gamma = ac/(a + c)$ from Eq. 4 gives $\gamma_{\text{base}} = 0.237$ and $\gamma_{\text{SMELT}} = 0.250$. The 5.5% higher γ means SMELT’s frontier loss drops faster per unit compute. Figure 4 confirms this visually: after subtracting each architecture’s own E , SMELT’s reducible-loss curves are steeper at every scale and sparsity.

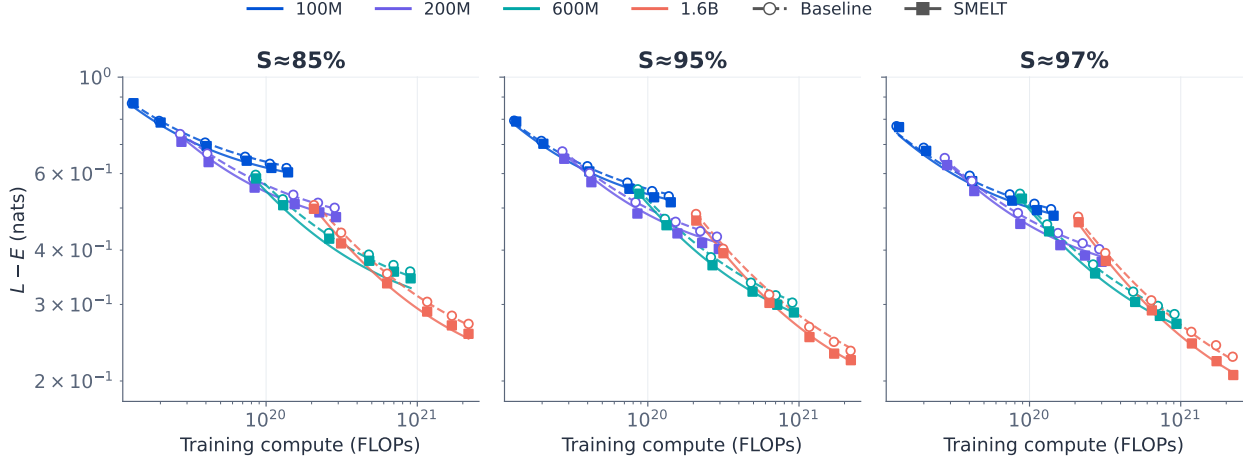


Figure 4 Reducible loss $\mathcal{L} - E$, where E is each architecture’s own irreducible loss, vs. training compute on the sparse scaling-law grid $S \approx \{85\%, 95\%, 97\%\}$. The $S = 0\%$ dense-reference control is omitted because its distinct regime is excluded from the sparse-grid fit. At every scale and sparsity, SMELT’s reducible loss decreases faster with compute.

Note that E_{SMELT} is slightly higher than E_{base} in Table 6, but the gap (0.005 nats) is smaller than the fit RMSE of either architecture, so we attribute it to fitting noise rather than a genuine difference in asymptotic floors. Predicting frontier behavior at much larger budgets (e.g. $C = 10^{25}$) would require training runs at those scales to anchor the fit.

CE Gain on the frontier. The steeper frontier translates into concrete compute savings. Table 7 applies the CE Gain procedure from Section 4.2 at three compute budgets. Its brackets come from a *cell bootstrap*: we resample the twelve sparse grid cells with replacement, refit both surfaces on each resample, recompute the quantity of interest, and report the 2.5th and 97.5th percentiles over 2,000 draws.

Table 7 CE Gain on the compute-optimal frontier: the fraction of training compute SMELT saves at the same loss. Brackets are cell-bootstrap 95% intervals (2,000 draws).

SMELT budget C_{tgt}	$S \approx 85\%$	$S \approx 95\%$	$S \approx 97\%$
10^{20} FLOPs	10.0% [1, 22]	7.8% [3, 15]	6.8% [4, 14]
10^{21} FLOPs	18.0% [8, 28]	15.8% [10, 25]	14.7% [8, 25]
10^{22} FLOPs [†]	23.5% [8, 42]	20.9% [0, 48]	19.6% [0, 51]

[†] Extrapolated beyond the fitted compute window.

At $C = 10^{20}$, SMELT saves 6.8–10.0% of training compute across sparsity levels. At $C = 10^{21}$, the range widens to 14.7–18.0%. Because $\gamma_{\text{SMELT}} > \gamma_{\text{base}}$, the gap between the two frontiers compounds over the fitted compute range at all three sparsities (the additive increment from 10^{20} to 10^{21} is ~ 8 pp in each column). S affects the level of the gain rather than its growth rate: $S \approx 85\%$ has the largest CE Gain because the sparsity term $(1-S)^b$ amplifies the gap, not because the gap widens faster.

Compute-optimal allocation. The separate surfaces also let us ask whether SMELT should allocate compute differently from the Baseline. From Eq. 3, the optimal per-token FLOPs scale as $F^* \propto C^{c/(a+c)}$. A larger

$c/(a+c)$ means a larger share of budget goes to model size rather than to tokens. We measure this through tokens per dense-equivalent parameter (TPP = $D^*/(F^*/6)$), where $F/6$ converts per-token FLOPs to an approximate active parameter count via the standard 6 FLOPs-per-parameter-per-token ratio [33]. Table 8 reports TPP at $C = 10^{21}$. Uncertainty comes from the same cell bootstrap as Table 7, here at 200 draws because each draw refits both surfaces and then re-solves the allocation.

Table 8 Compute-optimal tokens per dense-equivalent parameter (TPP) at $C = 10^{21}$. Brackets are cell-bootstrap 95% intervals.

S	Baseline TPP	SMELT TPP
$\approx 85\%$	56 [51, 65]	56 [44, 77]
$\approx 95\%$	78 [71, 90]	75 [60, 101]
$\approx 97\%$	91 [83, 104]	86 [70, 114]

The two architectures want nearly the same split: SMELT’s TPP point estimate is within 6% of the Baseline’s at every sparsity, and the bootstrap intervals overlap almost completely (Table 8). The data term in Table 6 explains why the two stay close. SMELT has both a larger data coefficient K (5.264×10^6 vs. 1.975×10^6) and a larger data exponent c (0.7011 vs. 0.6594). The larger K makes SMELT’s data-limited loss K/D^c higher at small D , below the crossover at $D \approx 1.6 \times 10^{10}$ tokens. The larger c makes the term fall faster as D grows. The two effects pull the optimal split in opposite directions and roughly cancel, so SMELT’s compute saving comes from reaching a lower loss at the same allocation rather than from reallocating the budget. Both architectures sit well above the ~ 20 TPP that Chinchilla reports for dense models [28]; sparse MoE models generally prefer more tokens per active parameter, consistent with prior work [1].

5 Downstream Performance and Domain Analysis

The CE Gain in Section 4 comes from validation loss, a single number that averages over all domains and sample lengths. A lower validation loss does not guarantee better downstream performance, and an aggregate improvement may hide biases across domains or sample types. This section tests whether the advantage transfers to independent benchmarks and examines how it distributes across training domains, sample lengths, and in-context learning.

5.1 Downstream performance amplifies the validation loss gap

SMELT wins on DCLM Completion in **96 of 96** matched pairs, on DCLM Core in 83 of 96, and on MMLU in 29 of the 30 pairs whose Baseline scores at least 10 percentage points above chance (Figures 5–7).

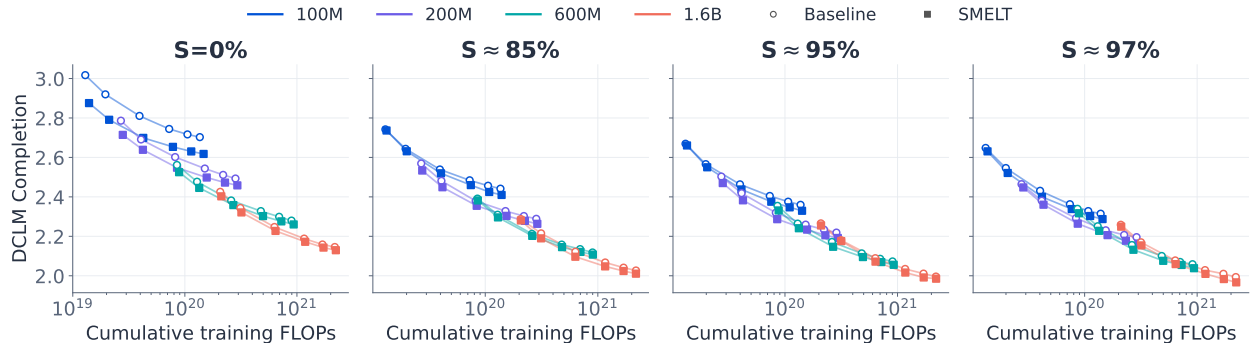


Figure 5 DCLM Completion (gold-completion micro loss) vs. cumulative training FLOPs across compute-equivalent sparsity levels. Lower is better; SMELT wins all 96 matched pairs.

Residual analysis against the Baseline curve. A model with lower validation loss should score better on any benchmark that tracks validation loss, so these win rates restate Section 4 rather than add to it. The key

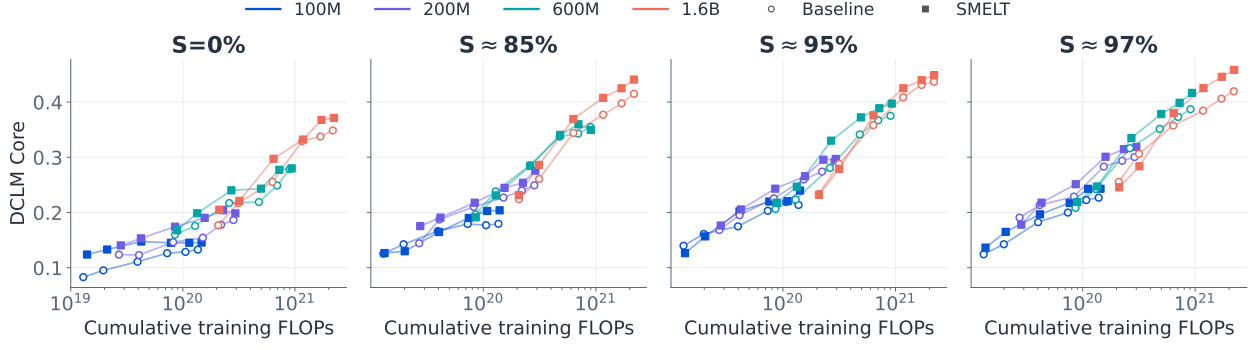


Figure 6 DCLM Core vs. cumulative training FLOPs across compute-equivalent sparsity levels. Higher is better; SMELT wins 83 of 96 matched pairs.

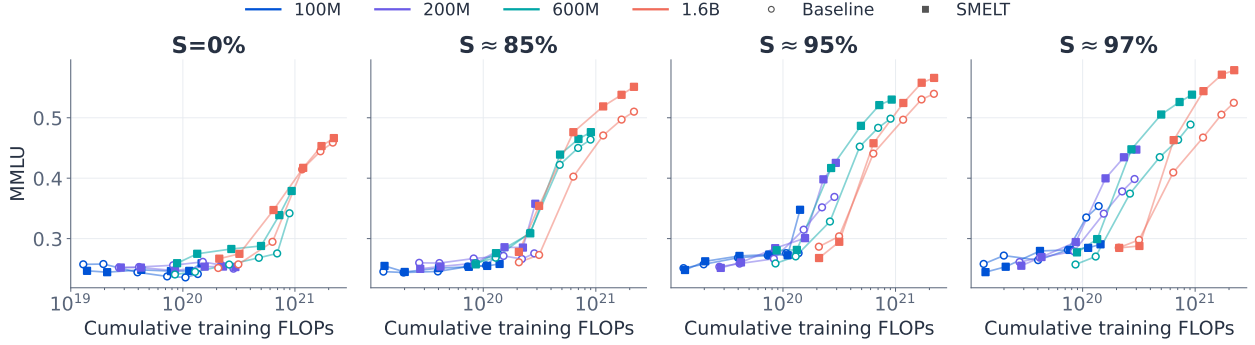


Figure 7 MMLU accuracy vs. cumulative training FLOPs across compute-equivalent sparsity levels; higher is better. The dotted line marks chance accuracy (25%). All 96 pairs are shown; because pairwise comparisons are unstable near the chance floor, the win-rate summary is restricted to the 30 pairs whose Baseline is at least 10 percentage points above chance, of which SMELT wins 29.

question is whether the downstream improvement is proportional to the validation loss reduction or exceeds it. To test this, we fit a calibration from validation loss ℓ to each benchmark score y , using all 96 Baseline endpoints as training data. Following prior task-scaling work that models loss-to-accuracy mappings with a sigmoid [7], we use the same four-parameter monotone sigmoid family for all three metrics:

$$\hat{y}_m(\ell) = b_m + \frac{a_m}{1 + \exp[-d_m \alpha_m (\ell - \tau_m)]}, \quad a_m, \alpha_m > 0, \quad (6)$$

where $d_m = +1$ for DCLM Completion, whose loss increases with validation loss, and $d_m = -1$ for the higher-is-better DCLM Core and MMLU scores; $a_m, b_m, \alpha_m, \tau_m$ are fitted separately for each metric. For the category breakdown in Figure 9b, we likewise fit the decreasing form ($d_m = -1$) independently to each of the 22 DCLM tasks before averaging the chance-centered residuals within each category. The fits are tight: $R^2 = 0.997$ for DCLM Completion, 0.974 for DCLM Core, and 0.911 for MMLU. For each SMELT endpoint i with validation loss ℓ_i and measured benchmark score y_i , we define the residual

$$\delta_i = s \cdot (y_i - \hat{y}(\ell_i)), \quad (7)$$

where $s = +1$ for higher-is-better metrics and $s = -1$ for DCLM Completion (a loss), so that a positive δ_i always means SMELT outperforms a Baseline that reaches the same validation loss.

The residual is positive on all three benchmarks and the excess grows with scale. Figure 8 shows that SMELT points separate from the Baseline curve favorably (below for DCLM Completion, above for the other two); Figure 9a breaks down the mean residual by scale. The mean residual $\bar{\delta}$ is positive on every benchmark at

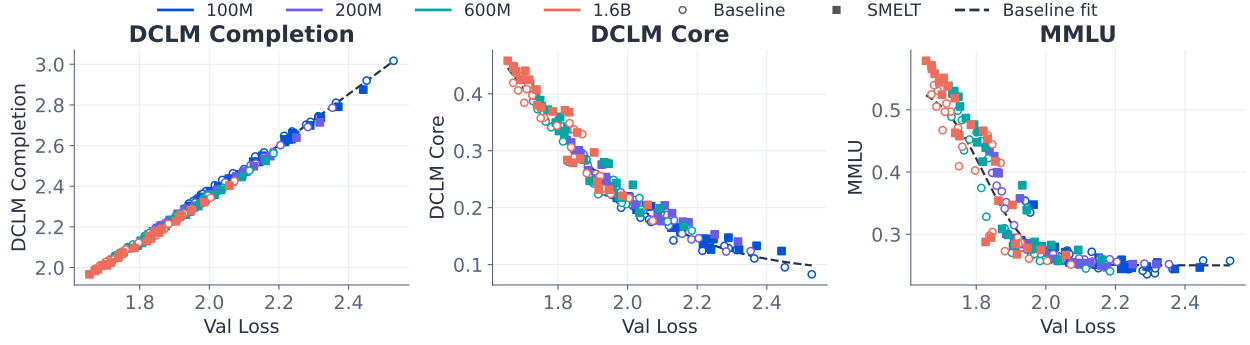


Figure 8 Benchmark scores against validation loss. DCLM Completion is a loss (lower is better); DCLM Core and MMLU are scores (higher is better). The curve fitted to the 96 Baseline endpoints uses the monotone sigmoid family in Eq. 6 for all three metrics ($R^2 = 0.997, 0.974, 0.911$ from left to right). At every scale on average, SMELT lies below the Baseline curve on DCLM Completion and above it on DCLM Core and MMLU: its downstream advantage exceeds what its validation-loss improvement alone predicts.

every scale. More importantly, $\bar{\delta}$ increases monotonically with model scale on DCLM Completion and DCLM Core; on MMLU the two larger scales exceed the two smaller ones. SMELT’s downstream advantage therefore exceeds what its validation loss improvement accounts for, and this excess grows as models scale up. Figure 9b further breaks down the residual by DCLM domain category (Appendix E). Reading Comprehension shows the largest excess when all scales are pooled, but the dominant category shifts as models grow, and Symbolic Problem Solving overtakes it at 1.6B. This suggests that the second visit increasingly benefits structured reasoning as model capacity grows.

	DCLM Completion (mn)	DCLM Core (pp)	MMLU (pp)		Reading Compreh.	World Knowledge	Common- sense	Language Underst.	Symbolic Problem
100M	+1.7 mn	+0.5 pp	+0.3 pp	100M	+4.7 pp	+0.7 pp	-0.6 pp	+1.0 pp	+0.3 pp
200M	+6.5 mn	+0.9 pp	+1.0 pp	200M	+1.7 pp	+0.2 pp	+0.4 pp	+0.7 pp	0.0 pp
600M	+11.6 mn	+1.0 pp	+2.2 pp	600M	+1.5 pp	+0.3 pp	+2.4 pp	+0.4 pp	+0.9 pp
1.6B	+12.3 mn	+1.2 pp	+1.6 pp	1.6B	+1.3 pp	+0.1 pp	+1.0 pp	+1.0 pp	+1.9 pp
All	+8.0 mn	+0.9 pp	+1.3 pp	All	+2.3 pp	+0.3 pp	+0.8 pp	+0.7 pp	+0.7 pp

(a) By aggregate benchmark. (b) By domain category.

Figure 9 Downstream residual of SMELT beyond what the validation loss improvement predicts (Eq. 7), broken down by model scale. In panel (a), the mean residual is positive on every benchmark at every scale; it increases monotonically with scale on DCLM Completion and DCLM Core, while the two larger scales exceed the two smaller ones on MMLU. mn = millinats (10^{-3} nats); pp = percentage points; DCLM Core and the category breakdown use centered accuracy (the DCLM-v2 reference baseline maps to zero), while MMLU uses raw accuracy. Panel (b) fits the same sigmoid separately to each DCLM task before category aggregation. The “All” row pools residuals over endpoints from all four scales rather than fitting a separate row. Positive residuals mean SMELT outperforms the Baseline-fitted curve; negative residuals mean it underperforms that curve.

5.2 SMELT gains most on structured data

The CE Gain in Section 4.3 averages over all validation domains. We now ask whether the gain is uniform across data types or concentrated on specific domains.

Method. To compute CE Gain on a subset of the data, we need a scaling surface restricted to that subset. A single subset does not have enough data to independently fit all six coefficients in Eq. 2. We therefore share the exponents (a, b, c) from the aggregate fit (Table 6) and fit only the intercepts (E_s, A_s, K_s) per subset, separately for the Baseline and for SMELT. This lets each subset converge to its own entropy floor while borrowing curvature from the data-rich aggregate fit. We then apply the CE Gain inversion from Section 4.2 at $C = 10^{21}$ and $S \approx 95\%$. We apply this procedure under two different groupings.

By domain category. The held-out validation set spans five categories (Section 3.1): Code, Math/STEM, Finance, Knowledge, and Web. We fit per-category intercepts and compute CE Gain for each.

By Baseline validation loss. Instead of using human-defined categories, we can also group sources by how well the Baseline already models them. Here we fit intercepts per individual source rather than per group, which gives a CE Gain for each of the 39 sources. Within each matched pair we then rank the 39 sources by the Baseline’s per-source validation loss and split them into four near-equal groups (10, 10, 10, 9): Q1 holds the sources with the lowest Baseline loss (data the Baseline already predicts well) and Q4 the highest. The CE Gain of each source is fixed (determined by the per-source intercept fit), but its quartile assignment can change from pair to pair because the ranking is based on each pair’s Baseline loss; the average is taken within each quartile and then across all 96 pairs.

Results. Figure 10 shows CE Gain under both groupings.

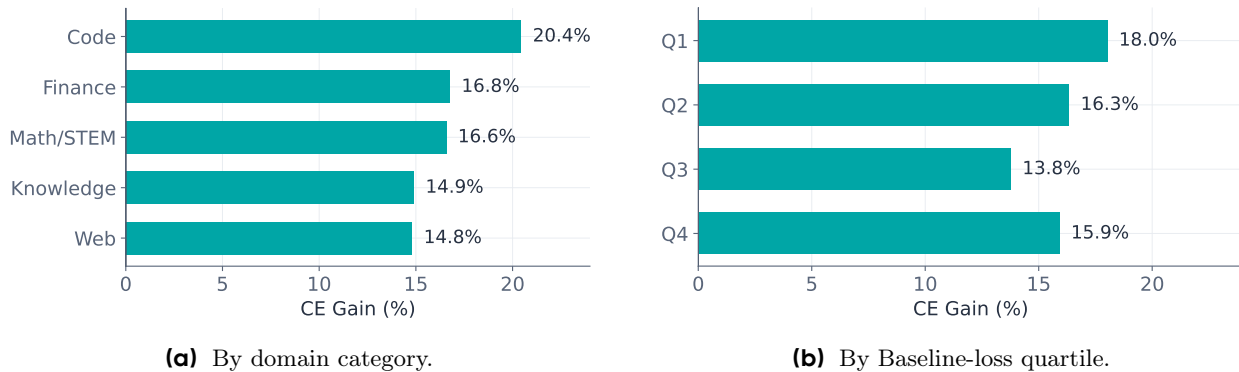


Figure 10 CE Gain on the compute-optimal frontier at $C = 10^{21}$ and $S \approx 95\%$. (a) From per-category intercepts: all five categories are positive and Code leads. (b) Within each matched pair, the 39 validation sources are ranked by Baseline per-source loss and split into four near-equal groups (10/10/10/9), from Q1 (lowest loss) to Q4 (highest). Averaged per-source CE Gain is highest in Q1, and the profile is U-shaped with its minimum at Q3.

CE Gain is positive on all five categories. Code leads at 20.4%; Finance and Math/STEM follow at 16.8% and 16.6%; Knowledge and Web sit at 14.9% and 14.8%. The ranking tracks the degree of internal structure: Code has strict syntax and long-range dependencies; Web text is less constrained.

On the Baseline-loss axis, CE Gain is not monotone: it falls from Q1 (18.0%) through Q2 (16.3%) to a minimum at Q3 (13.8%), then rebounds at Q4 (15.9%). The sources the Baseline already models well gain the most, but the hardest quartile is not the one that gains least. We read the Q1 end as the informative one: it lines up with the category breakdown, since Code has both the strongest structure and the lowest Baseline loss, and it has the highest CE Gain. Why the high-loss tail rebounds is not something our data settles; one possibility is that Q4 mixes genuinely noisy sources, where neither architecture can improve much, with hard-but-structured ones.

5.3 Gain grows with sample length and in-context examples

Section 5.2 groups the gain by data domain. We now examine two orthogonal dimensions: sample length and number of in-context examples.

SMELT’s gain concentrates on long samples. We evaluate per-token loss on the validation set, grouping samples into eight length buckets by each document’s own token count (upper bounds 32, 64, ..., 4096 tokens). We construct three families of pairwise contrasts and, for each pair, normalize the per-bucket improvement by its mean so that only the shape of the gain profile is compared:

- SMELT vs. Baseline: each SMELT run paired with its matched Baseline at the same scale and sparsity level;
- More experts: consecutive sparsity levels ($S = 0\% \rightarrow 85\%$, $85\% \rightarrow 95\%$, $95\% \rightarrow 97\%$) within the Baseline at the same model size;
- More parameters: consecutive active-parameter scales (e.g. 200M $\rightarrow$ 600M) within the Baseline at the same sparsity level.

Figure 11(a) plots the averaged normalized shape for each family. SMELT’s gain concentrates on long samples: the mean normalized gain over the four longest buckets (512–4096 tokens) is $1.52\times$ the mean over the four shortest (32–256 tokens). In contrast, the two Baseline families show no such tilt: adding parameters improves short and long samples roughly equally (ratio 0.98), and adding experts even favors short samples slightly (ratio 0.88).

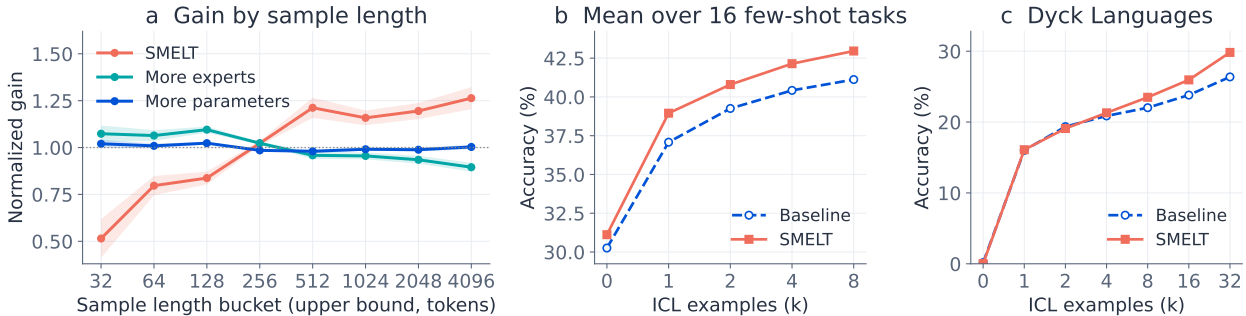


Figure 11 SMELT’s extra gain concentrates on long samples and on in-context learning. (a) Loss improvement by document length, normalized to mean 1 within each contrast so that the curves compare shape rather than magnitude. The three families are SMELT against its matched Baseline, and two Baseline-only controls that add experts or add active parameters. SMELT’s gain on the four longest buckets is $1.52\times$ its gain on the four shortest; the two controls stay flat (0.98 and 0.88). (b) Mean accuracy over 16 few-shot tasks (15 DCLM tasks plus MMLU) against the number of in-context examples k . The gap is 0.9 pp at $k = 0$ and 1.9 pp once demonstrations are available. (c) Dyck Languages task is demonstration-sensitive: both architectures score near 0% without examples, and the only way to improve is to read the demonstration answers. Here the gap keeps widening with k rather than plateauing, reaching 29.8% against 26.4% at $k = 32$.

SMELT benefits more from in-context examples. Figure 11(b) averages accuracy over 16 few-shot tasks, the 15 DCLM tasks that take demonstrations plus MMLU, as a function of the number of in-context examples k . This sweep is a separate evaluation from the aggregate metrics: it re-runs each task at $k \in \{0, 1, 2, 4, 8\}$ (and QA Wikidata, ARC Easy, CS Algorithms, Dyck Languages, and Operators additionally at $k = 16$ and 32) rather than at the fixed shot counts of Appendix E, which is why the accuracies here do not match the DCLM Core numbers elsewhere. The gap between the two architectures is 0.9 pp at $k = 0$ and widens to 1.9 pp once demonstrations are provided ($k = 1$); it then holds through $k = 8$. We highlight Dyck Languages (Figure 11(c)) because it is a demonstration-sensitive exact-match task: both architectures score near 0% without demonstrations, and the only way to improve is to read the example answers. On this task, SMELT reaches 29.8% at $k = 32$ vs. the Baseline’s 26.4%. On both the average and this demonstration-sensitive task,

SMELT gains more when in-context examples are available. Per-task accuracy curves for the 14 of these 16 tasks where SMELT’s max-shot gain is significant ($p < 0.05$, paired permutation test) are provided in Appendix C.

6 Inside the Second Pass

Sections 4–5 quantify the advantage of the second visit. To build intuition for what happens inside the model, we run all matched pairs on a 1M-token held-out validation sample and probe expert routing, residual-stream updates, and attention patterns, then ground the observations in a Dyck-language case study.

6.1 Do the two visits activate the same experts?

Each MoE layer selects a subset of experts for each token. When the same token passes through the same physical layer on the second visit, does the router pick the same experts, or does the updated residual-stream input lead it to a different subset?

Measurement. For each token and each repeated physical MoE layer, we record the top-8 expert sets selected by the gating network on visit 1 and visit 2, then count how many experts appear in both sets. This overlap count ranges from 0 (completely different routing) to 8 (identical routing). Figure 12 plots the distribution of this overlap count across all tokens at four sparsity levels: the $S=0\%$ dense-reference control and $S \approx \{85\%, 95\%, 97\%\}$.

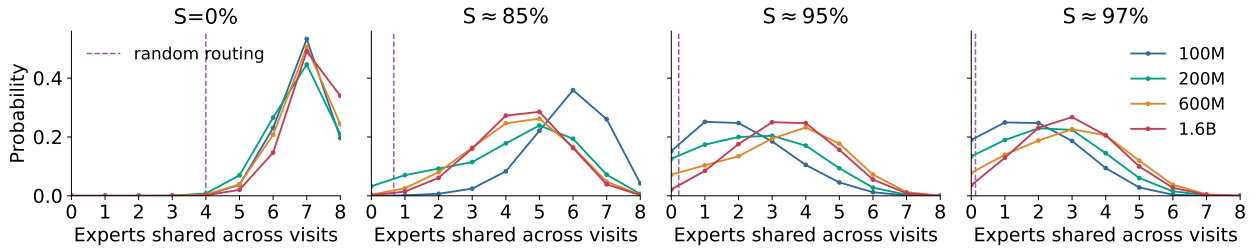


Figure 12 Distribution of experts shared between the two visits for the same token and physical gate. Four panels show the $S = 0\%$ dense-reference control and $S \approx \{85\%, 95\%, 97\%\}$. Dashed purple line: expected overlap under independent random routing ($8^2/n$ for top-8 routing over a candidate pool of n experts). At $S = 0\%$, the same token reuses nearly all 8 selected experts; at $S \approx 97\%$, it reuses only 2–3, still far above chance.

The router reuses a core subset and diversifies the rest. At $S=0\%$, where each layer has the smallest candidate pool in the grid, the router reuses nearly all 8. As sparsity increases and the pool grows by more than an order of magnitude, the overlap drops to 2–3. Even there it stays far above what independent random routing over a pool that large would give (dashed line in Figure 12), so the router deliberately reuses a core subset. The second visit reuses part of the first visit’s expert set and diversifies the rest.

6.2 Does the second visit write more or less into the residual stream?

Each Transformer layer updates the residual stream as $x_{\ell+1} = x_{\ell} + \Delta x_{\ell}$. When the same physical layer executes twice, is the second write $\Delta x_{\ell}^{(2)}$ larger or smaller than the first $\Delta x_{\ell}^{(1)}$? And do the two writes point in the same direction, or does the second visit write something orthogonal?

Setup. In a pre-norm Transformer, each layer decomposes into an attention write and an MoE write (we write LN for the pre-norm; our implementation uses RMSNorm [68]). Within the looped span, every sublayer’s residual update is scaled by $1/r$ where r is the loop count (Section 3.1):

$$\Delta x_{\ell}^{\text{attn}} = \frac{1}{r} \text{Attn}(\text{LN}(x_{\ell})), \quad (8)$$

$$\Delta x_{\ell}^{\text{moe}} = \frac{1}{r} \text{MoE}(\text{LN}(x_{\ell} + \Delta x_{\ell}^{\text{attn}})), \quad (9)$$

so the full layer update is $\Delta x_\ell = \Delta x_\ell^{\text{attn}} + \Delta x_\ell^{\text{moe}}$ and the MoE sub-layer sees the already-scaled attention write in its input. Outside the looped span $r = 1$, recovering the standard pre-norm form. For each of these three quantities, we measure the ℓ_2 norm on visit 1 and visit 2 and report the ratio $\|\Delta x_\ell^{(2)}\|/\|\Delta x_\ell^{(1)}\|$; a ratio above 1 means the second visit writes a larger update.

Visit 2 writes more, both absolutely and relatively. Figure 13 shows the layer-by-layer residual flow inside the repeated block at $S \approx 85\%$ across four scales. Row 2 plots the update norm $\|\Delta x_\ell\|$: at all four scales, visit 2 (red) writes a larger update than visit 1 (blue) at every layer. Row 3 normalizes by the residual itself, plotting $\|\Delta x_\ell\|/\|x_\ell\|$. At the three larger scales the relative update is also consistently higher on visit 2. At 100M, whose repeated block spans only five layers, a few early layers are comparable or reversed, likely because the residual has not yet diverged enough between visits.

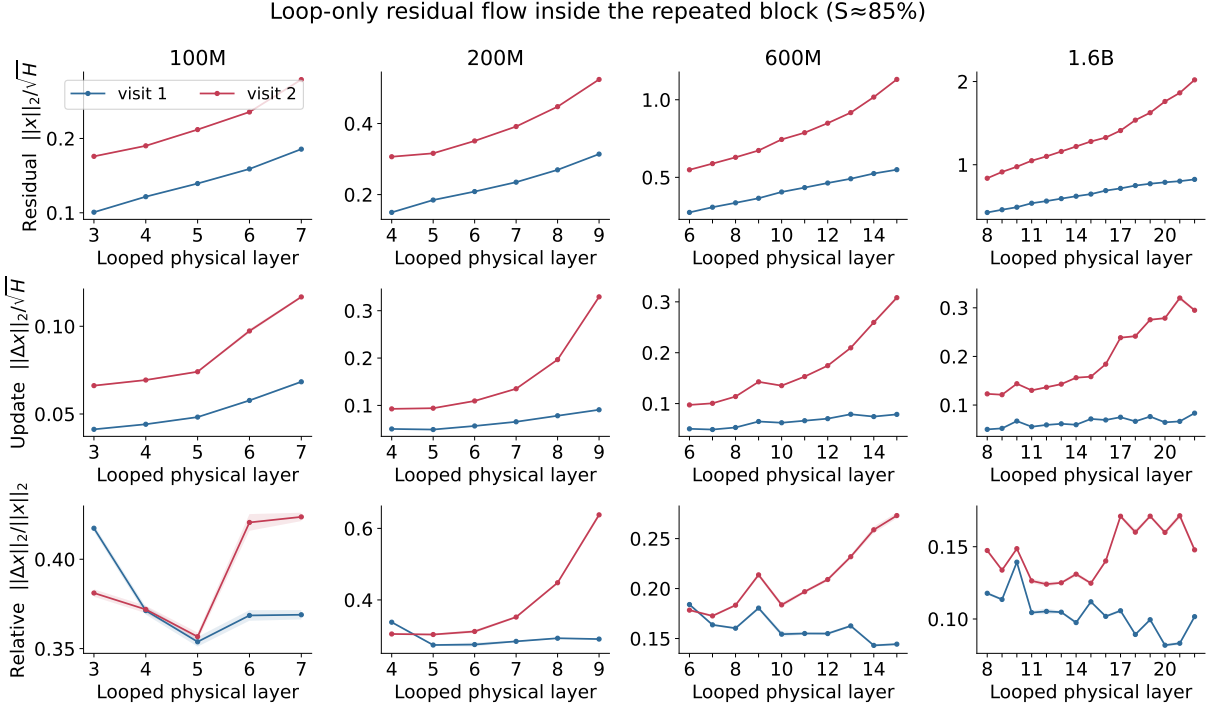


Figure 13 Residual flow inside the repeated block (matched $S \approx 85\%$, four scales). Row 1: residual RMS $\|x_\ell\|_2/\sqrt{H}$; row 2: update RMS $\|\Delta x_\ell\|_2/\sqrt{H}$; row 3: relative update $\|\Delta x_\ell\|_2/\|x_\ell\|_2$. Visit 2 (red) writes larger updates than visit 1 (blue) absolutely at every layer (row 2) and relative to the residual at the three larger scales (row 3).

Both attention and MoE sub-layers contribute. Where does the extra write come from, attention or MoE? Figure 14 collects eight cross-visit diagnostics over the 4×4 grid. Panels e–h each give the visit-2/visit-1 norm ratio of one component: the full update Δx_ℓ (panel e), the attention write $\Delta x_\ell^{\text{attn}}$ (panel f), the MoE write $\Delta x_\ell^{\text{moe}}$ (panel g), and the RMSNorm output that feeds the next sub-layer (panel h). Panels a–d carry the similarity measures of Sections 6.1–6.3 across the same grid, so the single-cell results in those sections can be checked at every scale and sparsity.

In all 16 cells, all four norm ratios exceed 1 (range 1.2–3.5 $\times$). Both the attention and MoE sub-layers contribute: panel h shows that the RMSNorm output fed to each sub-layer already has a larger norm on visit 2. Since RMSNorm removes input scale, this difference reflects a directional shift in the residual stream: the element-wise gain g amplifies different dimensions unequally, so a change in direction alone changes the output norm. Both sub-layer writes grow on top of this larger input.

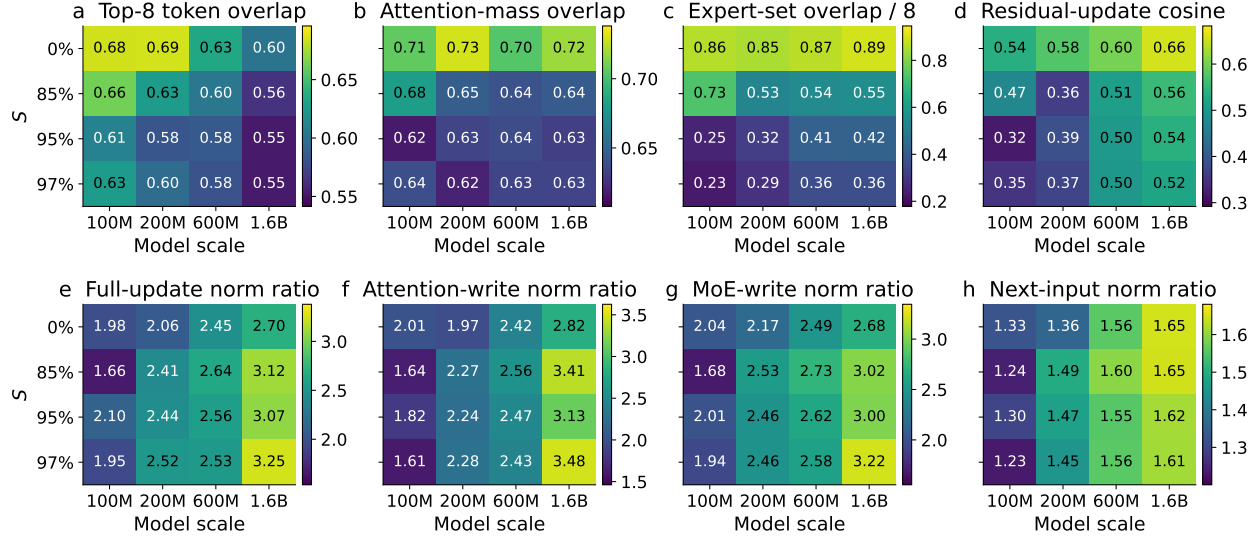


Figure 14 Eight cross-visit diagnostics over the 4×4 grid, with scale on the horizontal axis and compute-equivalent sparsity S on the vertical axis. (a)–(d) Similarity between the two visits: top-8 attended-token overlap, attention-mass overlap, expert-set overlap divided by 8, and residual-update cosine. (e)–(h) Visit-2 / visit-1 norm ratios: the full update Δx_ℓ , the attention write $\Delta x_\ell^{\text{attn}}$, the MoE write $\Delta x_\ell^{\text{moe}}$, and the RMSNorm output that feeds the next sub-layer. All four norm ratios exceed 1 in all 16 cells: the second visit writes more in every component, at every scale and sparsity.

The two writes are aligned. Knowing that the second visit writes more, we next ask whether it writes in the same direction. We compute the pairwise cosine similarity between residual updates Δx_ℓ across all layers. Figure 15 shows the resulting matrix for the 1.6B model: the Baseline on the left (30 unique layers), SMELT on the right (flattened execution order).

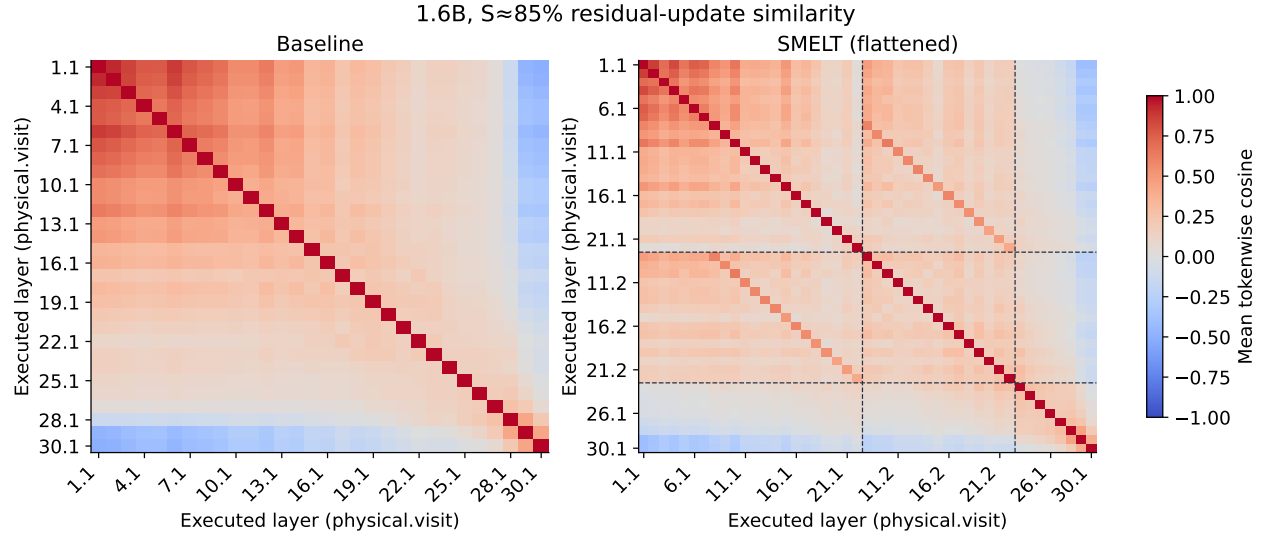


Figure 15 Pairwise residual-update cosine for the 1.6B model. The Baseline panel (left; 30 layers) provides the unlooped reference for ordinary layer-to-layer cosine structure. The SMELT panel (right) uses flattened execution order; dashed lines mark the repeated block. Along the same-physical-layer cross-visit diagonal, cosine ranges from 0.42 to 0.65 (mean 0.56), compared with a mean of 0.16 for nonmatching cross-visit pairs: the two visits write in more closely aligned directions at the same physical layer.

The cross-visit block (same physical layer, two visits) has positive cosine: the second visit writes in a similar direction to the first. The second visit does not override the first; it amplifies the signal the first visit established.

6.3 Does the second visit look at the same places or different ones?

The second visit recomputes Q , K , and V from the updated residual stream; it does not reuse cached activations from the first visit. Does this lead to different retrieval patterns (different Q and K , hence attending to different positions), or does the model preserve where it looks and only change what it reads?

Setup. Recall that the attention write from Eq. 8 decomposes as

$$\underbrace{\hat{x}_\ell}_X = \text{LN}(x_\ell), \quad Q = \hat{x}_\ell W_Q, \quad K = \hat{x}_\ell W_K, \quad V = \hat{x}_\ell W_V, \quad P = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \quad (10)$$

where Q and K set the retrieval coordinates (which positions to attend to), V carries the content at those positions, and P is the attention weight matrix. The attended context is PV and the final output written into the residual stream is $O = PVW_O$. For each of these six quantities (X , Q , K , V , PV , O), we compute the cosine similarity between visit 1 and visit 2 to see which stages change across visits and which stay fixed.

Q and K stay similar; V diverges. Figure 16 shows the result across the full 4×4 grid.

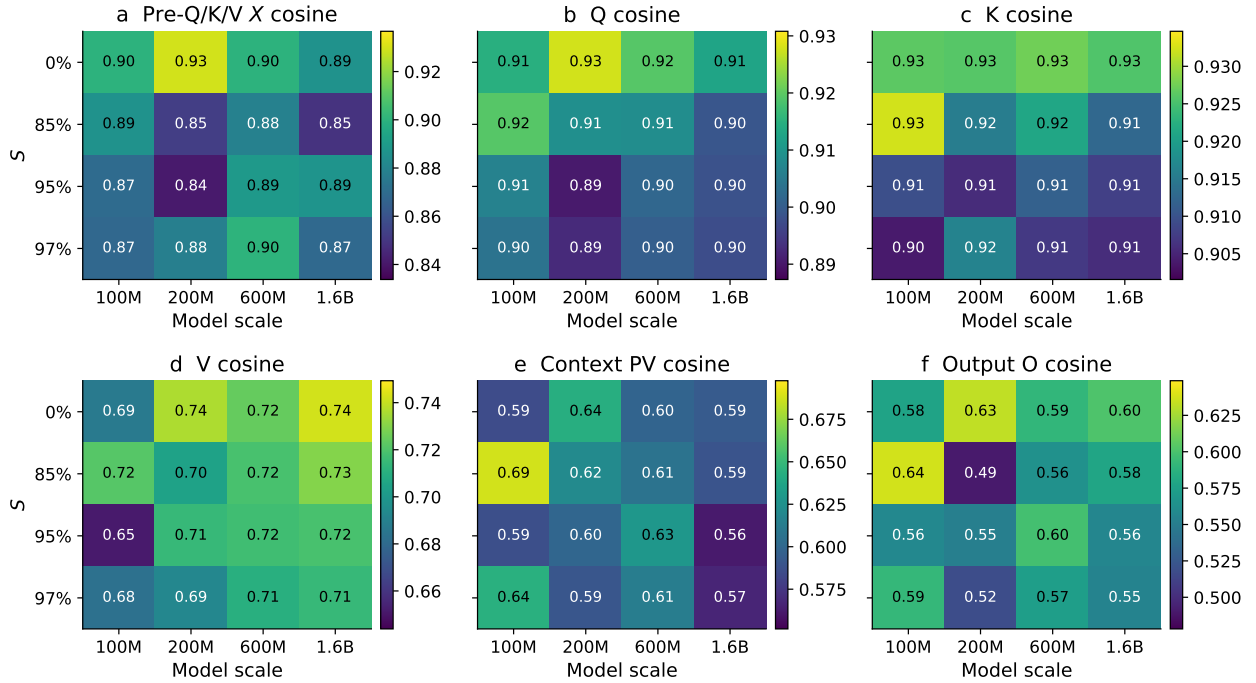


Figure 16 Cross-visit cosine similarity at six stages of attention, on the full 4×4 grid: (a) the shared pre-norm input X , (b) Q , (c) K , (d) V , (e) the attended context PV , and (f) the output O . Q and K stay at 0.89–0.93, at or above the input’s own 0.84–0.93, so the retrieval coordinates carry over. V falls below the input to 0.65–0.74, and PV and O diverge further.

The shared input X already has cross-visit cosine 0.84–0.93, which sets the reference level for the three projections that read it. Q and K sit at 0.89–0.93 across all 16 cells, at or above that level: the first visit establishes the retrieval coordinates and the second visit preserves them. V falls below it to 0.65–0.74, and the downstream stages PV and O diverge further (0.56–0.69 and 0.49–0.64). The divergence is therefore a

property of the value projection rather than of the input it reads: the two visits retain substantially overlapping attention patterns while V changes more than Q/K.

The two visits attend to nearly the same tokens. The Q/K similarity predicts that the two visits should attend to similar positions. We verify this at the discrete level: for each head, we record the top-8 attended tokens on each visit and measure set intersection. As a control, we pair each head with a different head at the same layer and visit, which shares the same positional context but uses different learned weights; the pairing is a fixed offset of half the head count, so every head is used exactly once. Figure 17 reports the result across all heads in the repeated block.

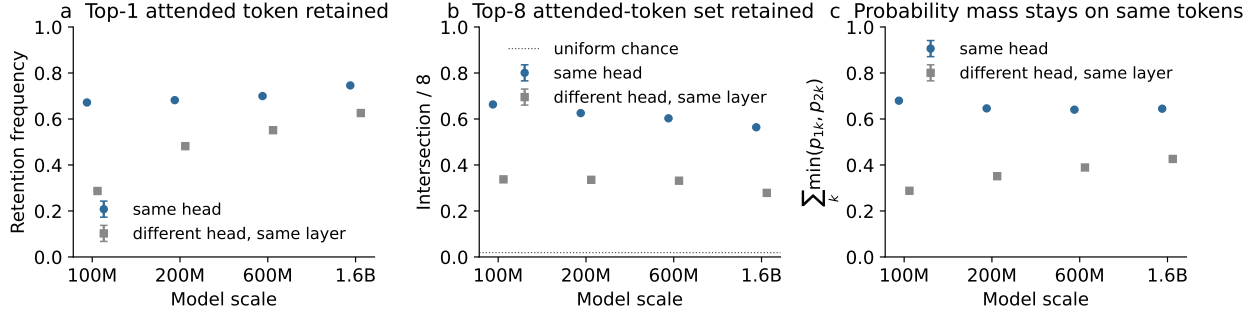


Figure 17 Attention token overlap between the two visits of the repeated block ($S \approx 85\%$, four scales): (a) top-1 retention, (b) top-8 set intersection, and (c) overlap of the full attention distributions. Blue compares the same head across visits; grey is a control that pairs each head with a different head in the same layer and visit, sharing the positional context but not the learned weights. The top-8 sets overlap by 56–66% across visits against 28–34% for the control, so the two visits attend to nearly the same tokens and the overlap is head-specific.

The top-8 attended tokens overlap by 56–66% across visits (Figure 17b). Selecting 8 out of up to 4,096 positions and recovering more than half the same set is a strong signal that the retrieval pattern is preserved. The other-head control sits at 28–34%, confirming that this overlap is head-specific, not an artifact of positional bias. The gap is narrower on top-1 retention (Figure 17a), where every head in a segment tends to land on the sink token and at the larger scales the control therefore starts high. Together with the Q/K–V split above, this section’s picture is: the two visits retain substantially overlapping attention patterns while V changes more than Q/K.

6.4 Case study: Dyck language

Section 5.3 shows that SMELT’s advantage grows with the number of in-context examples. To understand why, we study Dyck-language bracket matching, a demonstration-sensitive exact-match task where the correct answer is unambiguous: to close a bracket, the model must retrieve the matching opening symbol from a demonstration in the prompt. This lets us trace, at the individual token level, whether the second visit improves retrieval of the right demonstration.

Setup. We construct nested Dyck prompts at $k \in \{1, 2, 4, 8\}$ shots, with three fixed target positions whose correct closing symbol already appears in the $k \geq 1$ prefix. Each shot presents an input bracket sequence followed by its completion; we call the completion tokens that follow its “Output:” marker the *demonstration answer*. For each scale, we identify the *Dyck head*: we ablate each attention head in the repeated block one at a time and select the head whose removal increases Dyck perplexity the most. We then visualize this head’s full attention pattern to see where mass moves between the two visits.

The attention sink nearly vanishes on the second visit. Figure 18 shows the Dyck head’s attention map at 1.6B for a $k = 4$ example. On visit 1, the head concentrates mass on the attention sink at the segment-start BOS token and on nearby positions, as if the model does not yet know where to look. On visit 2, the sink nearly vanishes: mass shifts from BOS onto the demonstration answers in the prefix. The difference row

makes the redistribution explicit. Averaged over the three targets, BOS mass falls from 0.60 to 0.02 while the demonstration answers rise from 0.24 to 0.85.

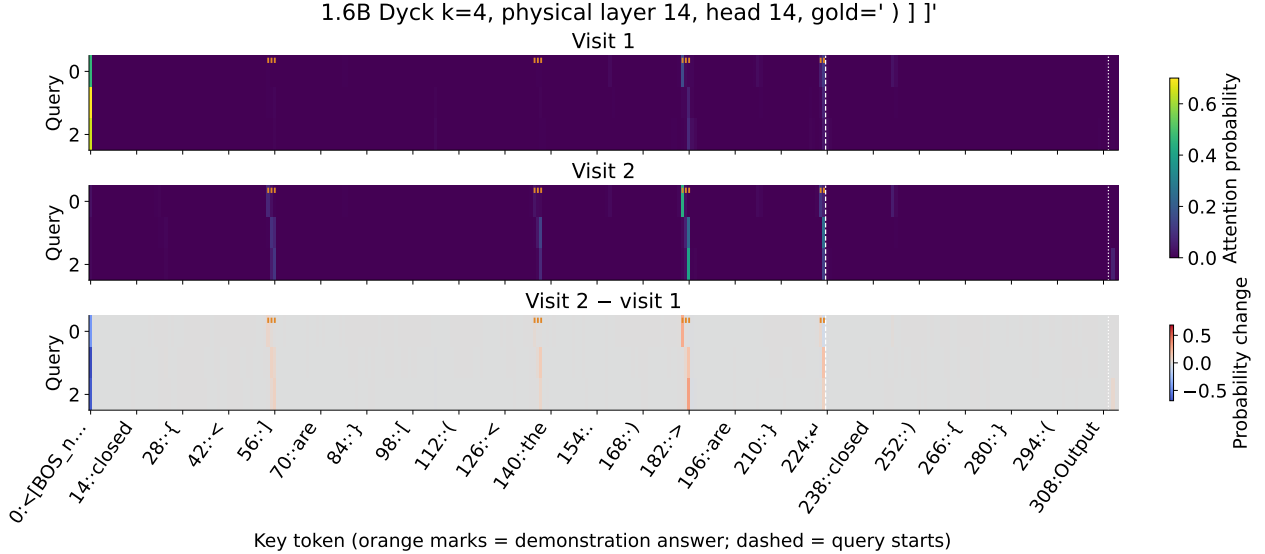


Figure 18 Token-level attention map for the *Dyck* head at 1.6B, $S \approx 85\%$, *Dyck* $k = 4$. The *Dyck* head is the head in the repeated block whose ablation raises *Dyck* perplexity the most, identified separately at each scale. Rows are the three prediction targets; columns are key positions. Top: visit 1; middle: visit 2; bottom: difference. Orange marks are the 11 demonstration-answer tokens of the four shots, the completions that follow each shot’s “Output:” marker. Averaged over the three targets, the second visit cuts BOS mass from 0.60 to 0.02 and lifts demonstration-answer mass from 0.24 to 0.85, so the sink loses almost exactly what the demonstrations gain.

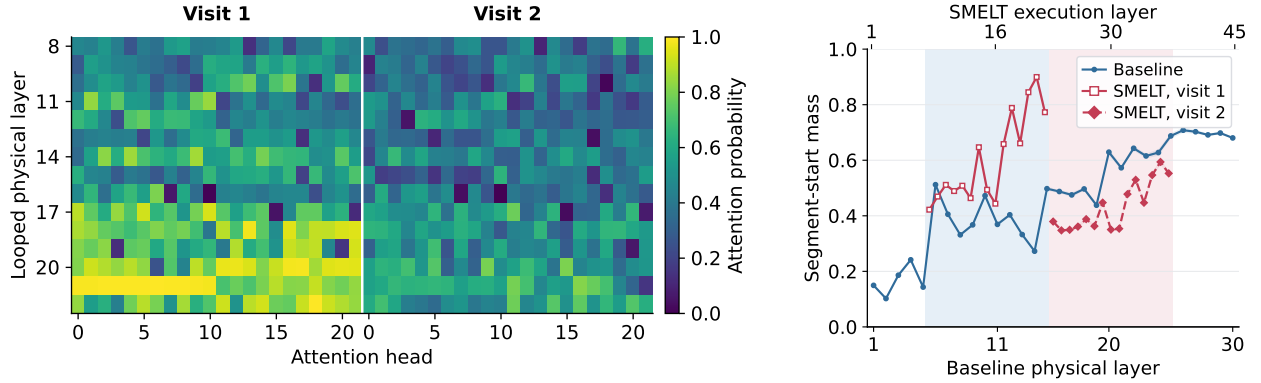
We suspect this pattern is related to the ICL advantage reported in Section 5.3: more demonstrations give the second visit more answer spans to attend to, and the sink reduction frees mass to reach them.

The sink reduction generalizes beyond Dyck. *Dyck* is a controlled task; does the same sink reduction occur on general pretraining data? We repeat the measurement on the full 1M-token held-out sample spanning all five source categories. The per-head heatmap (Figure 19a) shows that the bright, high-mass cells in visit 1 cool broadly on visit 2. We further find that this reduction goes against the ordinary depth trend. Figure 19b compares SMELT and the unlooped Baseline at the same physical depth: in the Baseline, sink mass grows toward later layers, consistent with the sink strengthening with depth in standard Transformers [6, 54], whereas SMELT’s visit 2 lies below visit 1 throughout the repeated block. The second visit thus reverses the depth trend instead of inheriting the larger sink that its later position in the execution trace would predict; Appendix D shows all four scales.

7 Conclusion and Future Work

We demonstrate that depth reuse is a net win for MoE Transformers under budget matching. By searching the loop configuration space under budget matching, we identify the SMELT recipe, which loops the middle half of layers twice at a larger effective depth-to-width ratio and consistently beats the Baseline across four scales and four sparsity levels, saving 6.8–18.0% of training compute on the frontier. The advantage extends beyond validation loss: on downstream benchmarks the gain exceeds what validation loss predicts, concentrates on structured data and long samples, and grows with in-context examples. Our mechanistic probes find a consistent internal signature: the second visit reuses retrieval coordinates, amplifies residual writes, and frees attention mass from the sink. This suggests that looping acts as a refinement step rather than simply adding capacity.

Several directions remain open. Due to compute constraints, we conduct the design ablations at the 200M scale



(a) Per-head segment-start attention mass at 1.6B. Visit 1 (left) concentrates mass on the segment start; visit 2 (right) reduces it across nearly every head.

(b) Sink mass at matched physical depth. The bottom axis indexes Baseline physical layers; the top axis indexes SMELT execution layers. The Baseline grows with depth; SMELT visit 2 stays below visit 1.

Figure 19 Segment-start attention mass (the attention sink) on the 1M-token held-out sample ($S \approx 85\%$). The second visit reduces the sink across nearly all heads and layers, running counter to the Baseline’s depth trend at matched physical depth.

(up to 3.9B non-embedding parameters); the optimal loop span or count may differ at larger scales. We also study only the simplest form of looping: repeating a contiguous block with fully shared weights. Richer variants relax this in different directions, including per-visit low-rank adapters [4], token-level adaptive recursion depth [5], learned halting [18, 19], block-selective sharing [32], and cross-token state reuse [9]. Whether these designs preserve or amplify the CE Gain under budget matching is left to future work. Separately, our budget matching equates arithmetic FLOPs, parameters, and KV-cache size, not wall-clock cost; serial block re-execution and sparse routing might introduce hardware-efficiency gaps that require systems-level optimization to close. Finally, understanding exactly which mechanisms inside the second visit are responsible for the gain remains an open question; our probes in Section 6 offer a descriptive starting point, and we expect future work to build on it toward a causal account.

Taken together, our results suggest that weight sharing across depth is a viable axis for improving language models at matched compute, complementary to the established axes of scaling width, depth, and expert count. We look forward to seeing how far this axis can be pushed as compute budgets and model scales continue to grow.

8 Contributions

Core Contributors

Shaowen Wang, Ge Zhang

Contributors

Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, Jian Li

Corresponding Authors

Shaowen Wang (wangsw23@mails.tsinghua.edu.cn)

Ge Zhang (gezhang@umich.edu)

Shen Yan (sheny@bytedance.com)

Jian Li (lijian83@mail.tsinghua.edu.cn)

Acknowledgments

We thank Kaifeng Lyu, Mianqiu Huang, Jinhan Li, Tansheng Zhu, Huaqing Zhang, Yilong Chen, Guancheng Du, Kexian Tang, Yihao Xiao, Zhiyuan Zeng, Jiameng Huang, Bingrui Li, Binghui Li, Li Cao, Zitian Gao, and Rui-Jie Zhu for their valuable input and helpful discussions.

The authors are supported in part by the National Natural Science Foundation of China Grant 04130200126.

References

- [1] Samira Abnar, Harshay Shah, Dan Busbridge, Alaaeldin El-Nouby, Joshua M. Susskind, and Vimal Thilak. Parameters vs FLOPs: Scaling laws for optimal sparsity for mixture-of-experts language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 204–230, 2025. URL <https://proceedings.mlr.press/v267/abnar25a.html>.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901. Association for Computational Linguistics, 2023. URL <https://doi.org/10.18653/v1/2023.emnlp-main.298>.
- [3] Ibrahim Alabdulmohsin and Xiaohua Zhai. Recursive inference scaling: A winning path to scalable inference in language and multimodal systems. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/9cf64a372841aee60c059923e3289048-Abstract-Conference.html.
- [4] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=WwpYS0kkCt>.
- [5] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron C. Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL http://papers.nips.cc/paper_files/paper/2025/hash/8b08bbf8b420faa6eeb4020720582ec7-Abstract-Conference.html.
- [6] Federico Barbero, Álvaro Arroyo, Xiangming Gu, Christos Perivolaropoulos, Michael M. Bronstein, Petar Veličković, and Razvan Pascanu. Why do LLMs attend to the first token? In *Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=tu4dFUw5z>.
- [7] Akshita Bhagia, Jiacheng Liu, Alexander Wettig, David Heineman, Oyvind Tafjord, Ananya Harsh Jha, Luca Soldaini, Noah A. Smith, Dirk Groeneveld, Pang Wei Koh, Jesse Dodge, and Hannaneh Hajishirzi. Establishing task scaling laws via compute-efficient model ladders. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=FeAM2RV081>.
- [8] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence*, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239.
- [9] Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T2MLR: Transformer with temporal middle-layer recurrence. In *Latent and Implicit Thinking Workshop at ICLR*, 2026. URL <https://openreview.net/forum?id=fQbk1EQWB0>.
- [10] Wenkai Chen, Tianshu Li, Wenyong Huang, Yichun Yin, Lifeng Shang, and Chengwei Qin. LoopMoE: Unifying iterative computation with mixture-of-experts for language modeling. *arXiv preprint arXiv:2606.04438*, 2026. URL <https://arxiv.org/abs/2606.04438>.
- [11] Yilong Chen, Naibin Gu, Junyuan Shang, Zhenyu Zhang, Yuchen Feng, Jiawei Sheng, Tingwen Liu, Shuohuan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Mixture of universal experts: Scaling virtual width via depth-width transformation. *arXiv preprint arXiv:2603.04971*, 2026. URL <https://arxiv.org/abs/2603.04971>.
- [12] Aidan Clark, Diego de Las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, George van den Driessche, Eliza Rutherford, Tom Hennigan, Matthew Johnson, Albin Cassirer, Chris Jones, Elena Buchatskaya, David Budden, Laurent Sifre,

- Simon Osindero, Oriol Vinyals, Marc’Aurelio Ranzato, Jack Rae, Erich Elsen, Koray Kavukcuoglu, and Karen Simonyan. Unified scaling laws for routed language models. In Proceedings of the 39th International Conference on Machine Learning, volume 162 of Proceedings of Machine Learning Research, pages 4057–4086, 2022. URL <https://proceedings.mlr.press/v162/clark22a.html>.
- [13] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 2924–2936, 2019. doi: 10.18653/v1/N19-1300. URL <https://aclanthology.org/N19-1300>.
 - [14] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. arXiv preprint arXiv:1803.05457, 2018. URL <https://arxiv.org/abs/1803.05457>.
 - [15] Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. MoEUT: Mixture-of-experts universal transformers. In Advances in Neural Information Processing Systems, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/321387ba926b8e58d3591c0aeb52ffc2-Abstract-Conference.html.
 - [16] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers. In International Conference on Learning Representations, 2024. URL <https://openreview.net/forum?id=2dn03LLiJ1>.
 - [17] Tom Davidson, Jean-Stanislas Denain, Pablo Villalobos, and Guillem Bas. AI capabilities can be significantly improved without expensive retraining. arXiv preprint arXiv:2312.07413, 2023. URL <https://arxiv.org/abs/2312.07413>.
 - [18] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In International Conference on Learning Representations, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
 - [19] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In International Conference on Learning Representations, 2020. URL <https://openreview.net/forum?id=SJg7KhVKPH>.
 - [20] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research, 23(120):1–39, 2022. URL <https://jmlr.org/papers/v23/21-0998.html>.
 - [21] Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, and Yu Wang. Think-at-hard: Dynamic looped transformers for improved reasoning. arXiv preprint arXiv:2511.08577, 2025. URL <https://arxiv.org/abs/2511.08577>. Accepted by ICML 2026.
 - [22] Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, and Bryan Dai. Loop the Loopies! arXiv preprint arXiv:2607.16051, 2026. URL <https://arxiv.org/abs/2607.16051>.
 - [23] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In Advances in Neural Information Processing Systems, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3b01972cf31e6fa0fe29e4b8b5c2a0a1-Abstract-Conference.html.
 - [24] Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. When attention sink emerges in language models: An empirical view. In International Conference on Learning Representations, 2025. URL <https://openreview.net/forum?id=78Nn4QJTEN>.
 - [25] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In International Conference on Learning Representations, 2021. URL <https://openreview.net/forum?id=d7KBjmI3GmQ>.
 - [26] Danny Hernandez and Tom B. Brown. Measuring the algorithmic efficiency of neural networks. arXiv preprint arXiv:2005.04305, 2020. URL <https://arxiv.org/abs/2005.04305>.
 - [27] Anson Ho, Tamay Besiroglu, Ege Erdil, Zifan Carl Guo, David Owen, Robi Rahman, David Atkinson, Neil Thompson, and Jaime Sevilla. Algorithmic progress in language models. In Advances in Neural

- Information Processing Systems, volume 37, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/6b066da6a23bc55f9b887e7298102884-Abstract-Conference.html.
- [28] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114ebe04a3e5-Abstract-Conference.html.
 - [29] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024. URL <https://arxiv.org/abs/2404.06395>.
 - [30] Martin Jaggi. Tying the loop – tied expert layers in mixture-of-experts language models, 2026. URL <https://arxiv.org/abs/2606.16825>.
 - [31] Kaggle. 200,000+ Jeopardy! questions. <https://www.kaggle.com/datasets/tunguz/200000-jeopardy-questions>, 2019. Dataset underlying the DCLM Jeopardy task; no accompanying publication.
 - [32] Ferdinand Kapl, Emmanouil Angelis, Kaitlin Maile, Johannes von Oswald, and Stefan Bauer. From growing to looping: A unified view of iterative computation in LLMs. *arXiv preprint arXiv:2602.16490*, 2026. URL <https://arxiv.org/abs/2602.16490>.
 - [33] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL <https://arxiv.org/abs/2001.08361>.
 - [34] Sanghoon Kim, Dahyun Kim, Chanjun Park, Wonsung Lee, Wonho Song, Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Lee, Jihoo Kim, Changbae Ahn, Seonghoon Yang, Sukyung Lee, Hyunbyung Park, Gyoungjin Gim, Mikyoung Cha, Hwalsuk Lee, and Sunghun Kim. SOLAR 10.7B: Scaling large language models with simple yet effective depth up-scaling. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 23–35, 2024. doi: 10.18653/v1/2024.naacl-industry.3. URL <https://aclanthology.org/2024.naacl-industry.3/>.
 - [35] Jonas Knupp, Jan Hendrik Metzen, Jeremias Bohn, Georg Groh, and Kristian Kersting. Depth-recurrent attention mixtures: Giving latent reasoning the attention it deserves. *arXiv preprint arXiv:2601.21582*, 2026. URL <https://arxiv.org/abs/2601.21582>.
 - [36] Vedang Lad, Wes Gurnee, and Max Tegmark. The remarkable robustness of LLMs: Stages of inference? *arXiv preprint arXiv:2406.19384*, 2024. URL <https://arxiv.org/abs/2406.19384>.
 - [37] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=H1eA7AEtvS>.
 - [38] Ryan Lee, Jacob Biloki, Edward J. Hu, and Jonathan May. Sparse layers are critical to scaling looped language models. *arXiv preprint arXiv:2605.09165*, 2026. URL <https://arxiv.org/abs/2605.09165>.
 - [39] Hector J. Levesque, Ernest Davis, and Leora Morgenstern. The Winograd schema challenge. In *Principles of Knowledge Representation and Reasoning: Proceedings of the Thirteenth International Conference (KR)*. AAAI Press, 2012. URL <https://aaai.org/papers/59-4492-the-winograd-schema-challenge/>.
 - [40] Houyi Li, Ka Man Lo, Shijie Xuyang, Ziqi Wang, Wenzhen Zheng, Haocheng Zhang, Zhao Li, Shuigeng Zhou, Xiangyu Zhang, and Daxin Jiang. Mixture-of-experts can surpass dense LLMs under strictly equal resource. *arXiv preprint arXiv:2506.12119*, 2025. URL <https://arxiv.org/abs/2506.12119>.
 - [41] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruva Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi

- Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. DataComp-LM: In search of the next generation of training sets for language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- [42] Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3EWTEy9MTM>.
- [43] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [44] Jan Ludziejewski, Jakub Krajewski, Kamil Adamczewski, Maciej Pióro, Michal Krutul, Szymon Antoniak, Kamil Ciebia, Krystian Król, Tomasz Odrzygóźdź, Piotr Sankowski, Marek Cygan, and Sebastian Jaszczur. Scaling laws for fine-grained mixture of experts. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 33270–33288, 2024. URL <https://proceedings.mlr.press/v235/ludziejewski24a.html>.
- [45] William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023. URL https://doi.org/10.1162/tac1_a_00562.
- [46] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, 2018. doi: 10.18653/v1/D18-1260. URL <https://aclanthology.org/D18-1260>.
- [47] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, 2016. doi: 10.18653/v1/P16-1144. URL <https://aclanthology.org/P16-1144>.
- [48] Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/b6341525cd84f3be0ef203e4d7cd8556-Abstract-Conference.html.
- [49] Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling laws for stable looped language models. *arXiv preprint arXiv:2604.12946*, 2026. URL <https://arxiv.org/abs/2604.12946>.
- [50] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/904e89bb4e632e75fb47f093b620b257-Paper-Conference.pdf.
- [51] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016. doi: 10.18653/v1/D16-1264. URL <https://aclanthology.org/D16-1264>.
- [52] Siva Reddy, Danqi Chen, and Christopher D. Manning. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019. doi: 10.1162/tac1_a_00266. URL <https://aclanthology.org/Q19-1016>.
- [53] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *Logical Formalizations of Commonsense Reasoning, Papers from the 2011 AAAI Spring Symposium*. AAAI, 2011. URL <http://www.aaai.org/ocs/index.php/SSS/SSS11/paper/view/2418>.

- [54] Valeria Ruscio, Umberto Nanni, and Fabrizio Silvestri. What are you sinking? A geometric approach on attention sink. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b2c4b7d34b3d96b9dc12f7bce424b7ae-Abstract-Conference.html.
- [55] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence*, pages 8732–8740. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6399.
- [56] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=din0lGfZFd>.
- [57] Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How much is one recurrence worth? iso-depth scaling laws for looped language models. *arXiv preprint arXiv:2604.21106*, 2026. URL <https://arxiv.org/abs/2604.21106>.
- [58] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=B1ckMDqlg>.
- [59] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=uyTL5Bvosj>.
- [60] Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024. URL <https://arxiv.org/abs/2402.17762>.
- [61] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4149–4158, 2019. doi: 10.18653/v1/N19-1421. URL <https://aclanthology.org/N19-1421>.
- [62] Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, 2019. doi: 10.18653/v1/P19-1452.
- [63] Howe Tissue, Venus Wang, and Lu Wang. Scaling law with learning rate annealing. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL http://papers.nips.cc/paper_files/paper/2025/hash/830b1abc6d2da85f23d41169fa44d185-Abstract-Conference.html.
- [64] Shaowen Wang, Bingrui Li, Ge Zhang, Wenhao Huang, Shen Yan, and Jian Li. On the residual scaling of looped transformers: Stability and transferability. *arXiv preprint arXiv:2606.18524*, 2026. URL <https://arxiv.org/abs/2606.18524>.
- [65] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- [66] Liu Yang, Kangwook Lee, Robert D. Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=HHbRxoDTxE>.
- [67] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472>.
- [68] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems* 32, pages 12360–12371, 2019.
- [69] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. AGIEval: A human-centric benchmark for evaluating foundation models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2299–2314, 2024. doi: 10.18653/v1/2024.findings-naacl.149. URL <https://aclanthology.org/2024.findings-naacl.149/>.

- [70] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew K. Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason K. Eshraghian. Scaling latent reasoning via looped language models. arXiv preprint arXiv:2510.25741, 2025. URL <https://arxiv.org/abs/2510.25741>.

Appendix

A Model Configurations

Tables 9 and 10 list the Baseline and SMELT configurations in the 4×4 grid. The Baseline configurations come from a tuned MoE family. Scale identifies each matched compute tier by its Baseline active-parameter scale. The Active params column gives the non-embedding parameter footprint of one physical traversal: attention, normalization, and router weights together with the selected top-8 expert weights. A shared loop block contributes once to Active params. The S column gives the measured compute-equivalent sparsity from Eq. 1, whose FLOPs F count every block execution; the paper refers to the three sparse levels by the integer labels $S \approx \{85\%, 95\%, 97\%\}$.

Implementation note. Our models are built on a proprietary Transformer family whose complete architecture and training stack cannot be released. We report the quantities directly relevant to the matched comparisons: model scale, hidden dimension, physical depth, expert count, active parameter count, non-embedding parameter count, and compute-equivalent sparsity. Both architectures are constructed using the same scaling rules and training stack; differences are restricted to the loop configuration and the budget-matching adjustments described in Section 3.2. For physical depth L and loop-span length $m = L/2$, we place $\lfloor (L - m)/2 \rfloor$ layers before the repeated span; with 1-based indexing, the looped layers are 3–7, 4–9, 6–15, and 8–22 for $L = 10, 12, 20$, and 30, respectively. Note that we use different H values at the 600M scale: $H = 1408$ for $S = 0$ and $S \approx 85\%$, and $H = 1512$ for $S \approx 95\%$ and 97% . This is because budget matching jointly adjusts H , the expert intermediate dimension, and attention-head geometry to obtain the closest match in per-token FLOPs, total parameters, and KV cache.

Table 9 Baseline configurations for all 16 grid cells. H is the hidden dimension and L the physical depth. Total experts and Active experts give the stored pool size and the number selected by each routing decision, respectively. Parameter counts exclude embeddings.

Scale	H	L	Total experts	Active experts	Active params	S	Total params
100M	672	10	8	8	0.100B	0.0%	0.10B
100M	672	10	64	8	0.100B	86.0%	0.71B
100M	672	10	192	8	0.101B	95.2%	2.12B
100M	672	10	336	8	0.102B	97.2%	3.70B
200M	1280	12	8	8	0.207B	0.0%	0.21B
200M	1280	12	64	8	0.207B	84.3%	1.32B
200M	1280	12	192	8	0.209B	94.6%	3.87B
200M	1280	12	336	8	0.212B	96.9%	6.74B
600M	1792	20	8	8	0.663B	0.0%	0.66B
600M	1792	20	64	8	0.665B	84.5%	4.28B
600M	1792	20	192	8	0.670B	94.7%	12.54B
600M	1792	20	336	8	0.675B	96.9%	21.83B
1.6B	2304	30	8	8	1.629B	0.0%	1.63B
1.6B	2304	30	64	8	1.632B	84.5%	10.55B
1.6B	2304	30	192	8	1.641B	94.7%	30.94B
1.6B	2304	30	336	8	1.651B	96.9%	53.89B

Table 10 SMELT configurations for all 16 grid cells. H is the hidden dimension and L the physical depth. Total experts and Active experts give the stored pool size and the number selected by each routing decision, respectively. Loop gives the repeated span and total number of passes. Parameter counts exclude embeddings. The final two columns give the per-token training-FLOPs and KV-cache ratios relative to the corresponding Baseline.

Scale	H	L	Total experts	Active experts	Loop Configuration	Active params	S	Total params	$F_{\text{SMELT}}/F_{\text{Base}}$	$KV_{\text{SMELT}}/KV_{\text{Base}}$
100M	576	10	16	8	Mid. 50% $\times 2$	0.071B	0.0%	0.10B	1.081	1.018
100M	576	10	96	8	Mid. 50% $\times 2$	0.067B	86.8%	0.71B	1.024	0.964
100M	576	10	288	8	Mid. 50% $\times 2$	0.069B	95.5%	2.12B	1.031	0.964
100M	576	10	504	8	Mid. 50% $\times 2$	0.070B	97.4%	3.70B	1.039	0.964
200M	1056	12	16	8	Mid. 50% $\times 2$	0.142B	0.0%	0.21B	1.039	1.031
200M	1056	12	96	8	Mid. 50% $\times 2$	0.140B	84.6%	1.32B	1.021	1.031
200M	1056	12	288	8	Mid. 50% $\times 2$	0.142B	94.7%	3.89B	1.029	1.031
200M	1056	12	504	8	Mid. 50% $\times 2$	0.145B	96.9%	6.78B	1.037	1.031
600M	1408	20	16	8	Mid. 50% $\times 2$	0.458B	0.0%	0.67B	1.046	1.031
600M	1408	20	96	8	Mid. 50% $\times 2$	0.443B	85.1%	4.31B	1.007	1.031
600M	1512	20	288	8	Mid. 50% $\times 2$	0.453B	94.7%	12.44B	1.024	0.984
600M	1512	20	504	8	Mid. 50% $\times 2$	0.459B	96.9%	21.70B	1.031	0.984
1.6B	1848	30	16	8	Mid. 50% $\times 2$	1.101B	0.0%	1.64B	1.022	0.984
1.6B	1848	30	96	8	Mid. 50% $\times 2$	1.081B	84.6%	10.45B	1.000	0.984
1.6B	1848	30	288	8	Mid. 50% $\times 2$	1.091B	94.8%	30.90B	1.004	0.984
1.6B	1848	30	504	8	Mid. 50% $\times 2$	1.103B	97.0%	53.90B	1.008	0.984

Matching granularity. The residual mismatch is larger at the least sparse reference level, where the expert pool is small and each hardware-aligned change to model width, expert intermediate dimension, head geometry, or expert count moves a comparatively large fraction of the budget. The feasible configurations therefore form a coarser grid than at the three sparse levels. Across the 12 sparse cells used in that fit, the maximum absolute mismatches are 3.9% in per-token FLOPs, 1.0% in total parameters, and 3.6% in KV cache. Besides, these residual differences are not treated as exact budget equality in the scaling analysis: every run enters the fit at its measured per-token FLOPs F , and the reported compute-efficiency comparisons evaluate the two separately fitted surfaces at common (C, S) coordinates.

B Training Curves for Every Grid Cell

Section 4.1 shows training curves for two representative cells. This appendix gives all 16 cells of the 4×4 grid: four scales crossed with the $S=0\%$ dense-reference control and three sparse levels $S \approx \{85\%, 95\%, 97\%\}$. Each panel plots training loss against cumulative training tokens for one matched Baseline / SMELT pair. Solid lines are the stable constant-learning-rate phase; dashed lines are the six cosine-decay branches forked at steps 10,000, 20,000, 50,000, 100,000, 150,000, and 196,075. SMELT ends below the Baseline in all 16 cells.

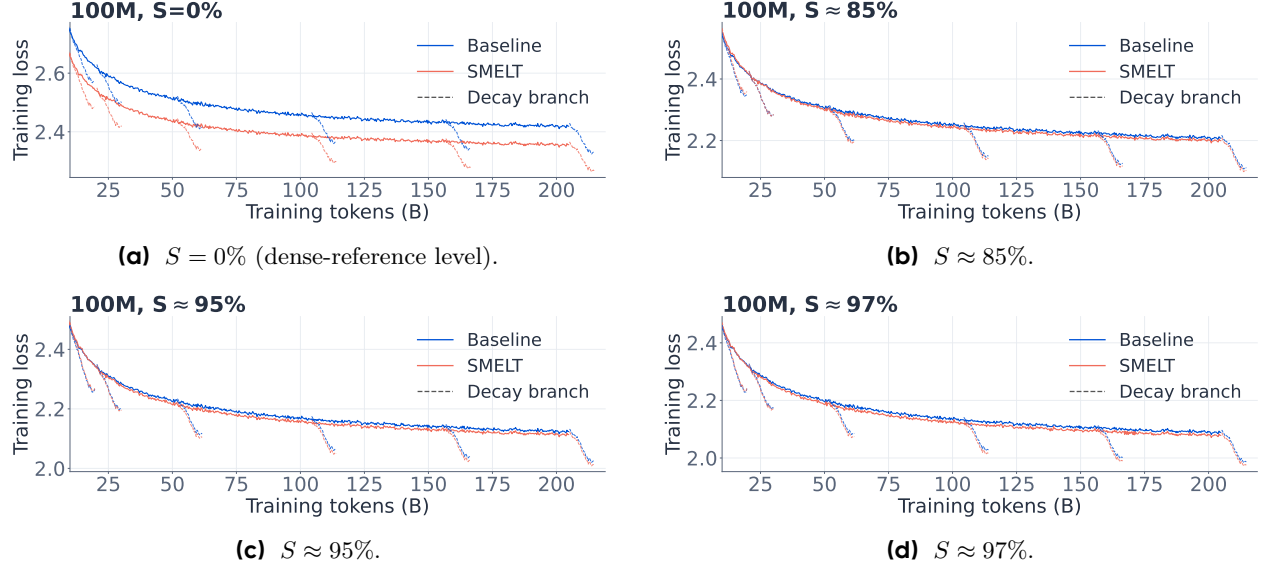


Figure 20 Training loss at the 100M scale across all four sparsity levels. The Baseline is shown in blue and SMELT in red.

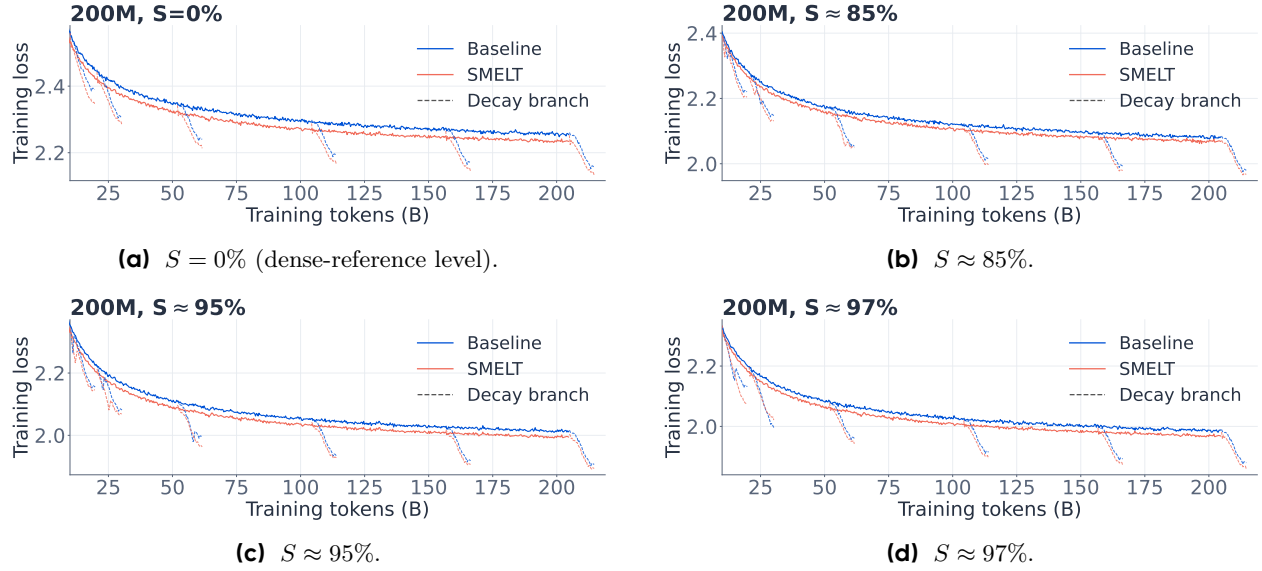


Figure 21 Training loss at the 200M scale across all four sparsity levels. The Baseline is shown in blue and SMELT in red.

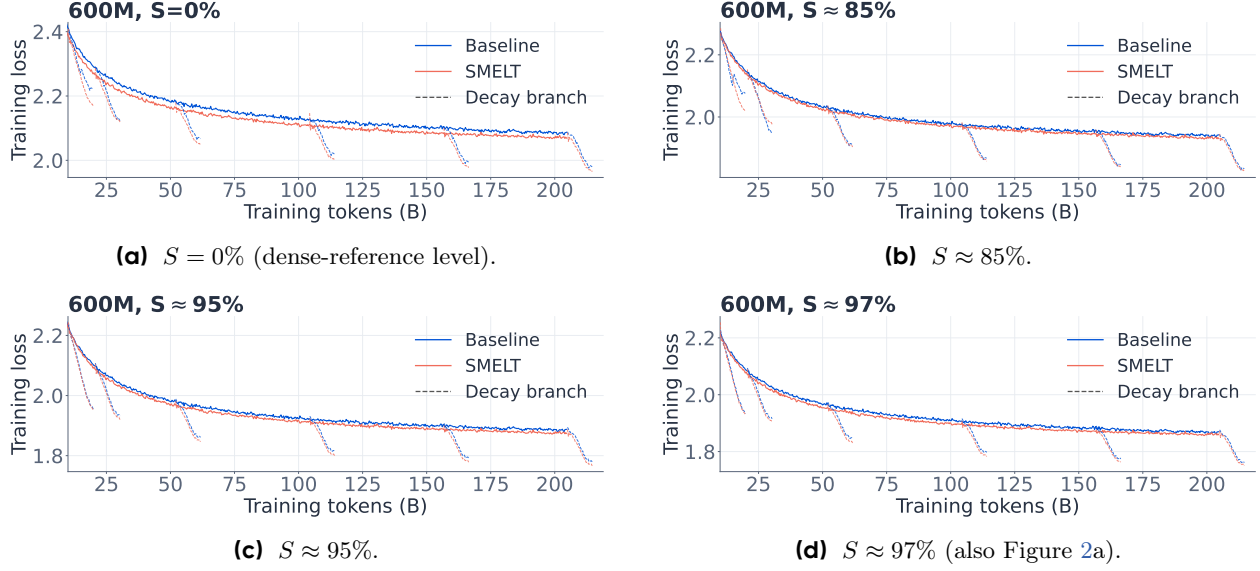


Figure 22 Training loss at the 600M scale across all four sparsity levels. The Baseline is shown in blue and SMELT in red.

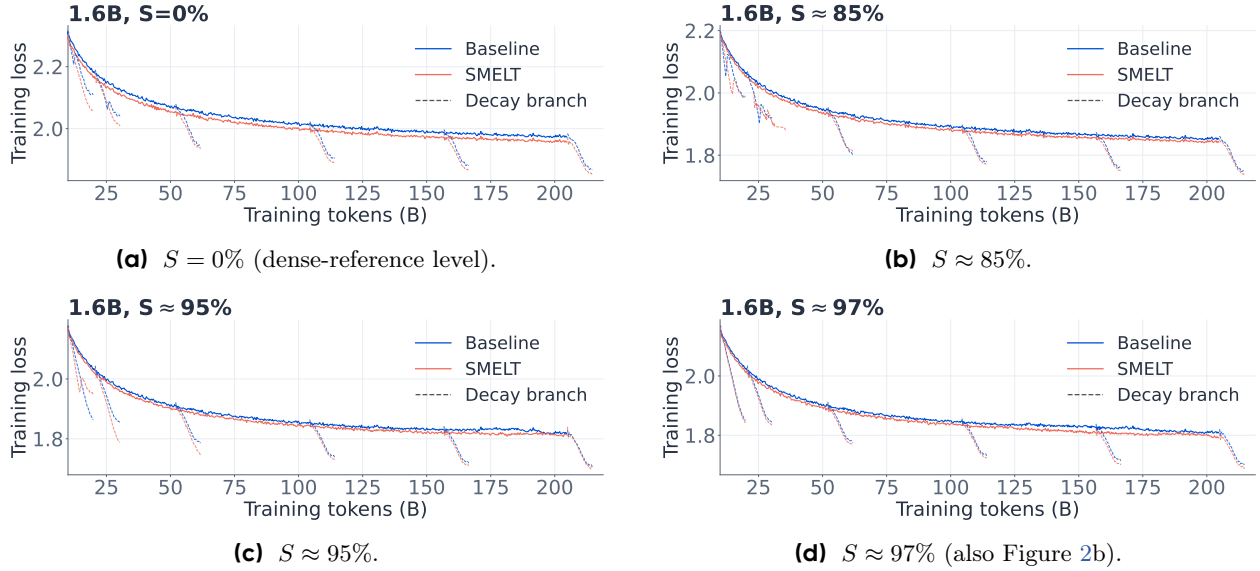


Figure 23 Training loss at the 1.6B scale across all four sparsity levels. The Baseline is shown in blue and SMELT in red.

C Per-Task ICL Accuracy Curves

The shot sweep of Section 5.3 covers 16 few-shot tasks: the 15 DCLM Core tasks that take in-context demonstrations, plus MMLU. Figure 24 shows accuracy as a function of the number of in-context examples k for the 14 of those 16 tasks where SMELT’s accuracy gain at the maximum shot count is positive with $p < 0.05$ under a paired permutation test over the 10 random seeds. Each curve averages first within each (scale, sparsity) grid cell where both architectures score above chance, then across cells, so that no single scale dominates.

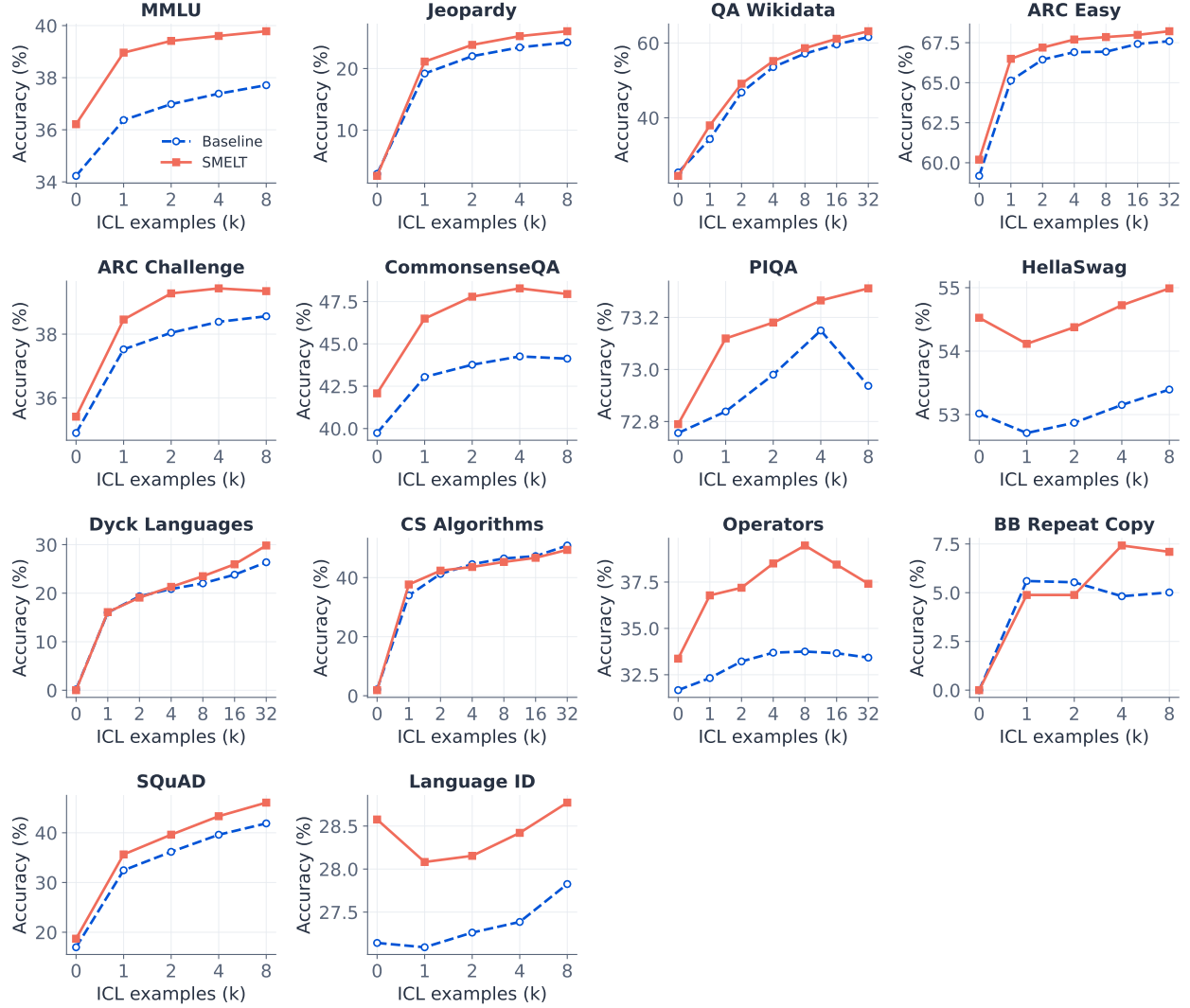


Figure 24 Per-task accuracy vs. number of in-context examples k for the 14 of 16 tasks with a significant max-shot gain ($p < 0.05$, paired permutation test). Baseline (dashed, blue) and SMELT (solid, red). Each curve averages first within each grid cell, then across cells.

D Attention-Sink Profiles Across Scales

Figure 25 expands the 1.6B comparison in Figure 19b to all four model scales. At each scale, the Baseline and SMELT have the same physical depth. The Baseline profile generally rises toward later layers, whereas SMELT’s visit 2 remains below visit 1 throughout the repeated block.

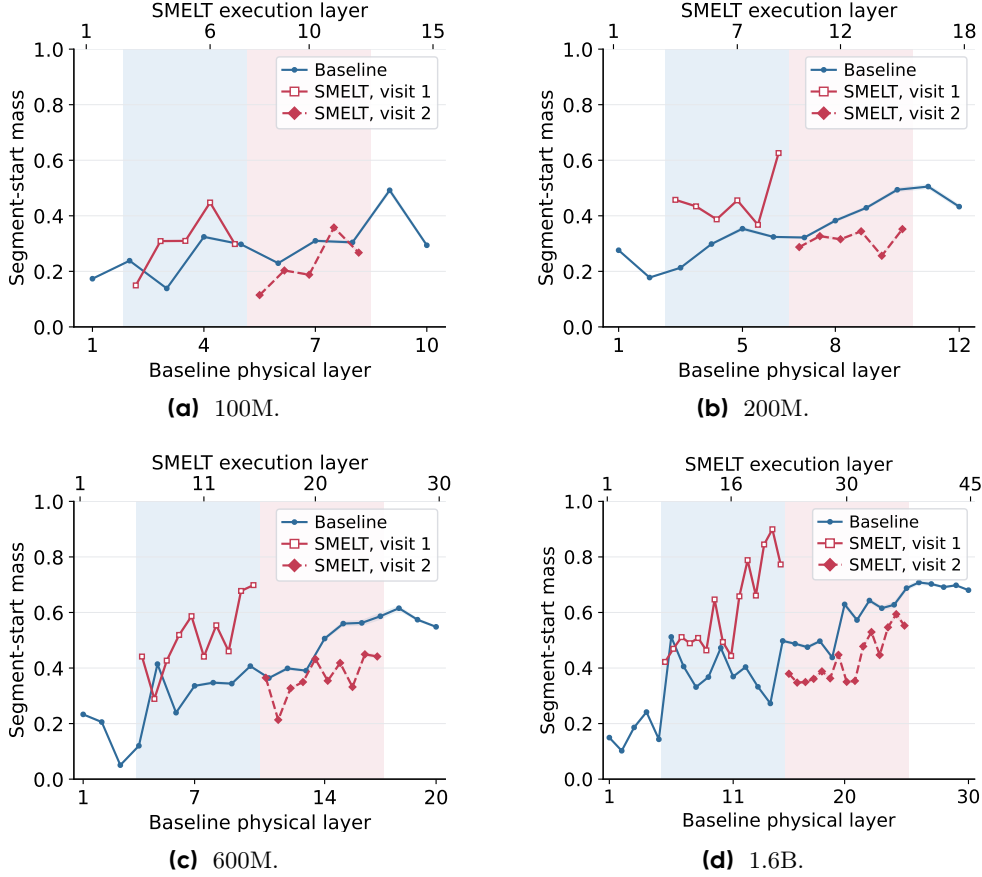


Figure 25 Same-physical-depth attention-sink profiles on the 1M-token held-out sample ($S \approx 85\%$). Blue circles show the Baseline; open red squares and filled red diamonds show SMELT visits 1 and 2. Bottom axes index Baseline physical layers and top axes index SMELT execution layers. Across all four scales, the second visit reduces segment-start mass even though the unlooped Baseline tends to accumulate more sink at later depth.

E Downstream Evaluation Protocol

This appendix defines the downstream metrics, multi-seed protocol, and task categorization referenced by the main text.

Task suite. We evaluate on the 22-task DCLM Core suite [41] and, separately, on MMLU 5-shot [25]. MMLU is not part of the official DCLM Core set but is widely used as a standalone benchmark; we report it individually rather than folding it into our aggregate metrics. We use the v2 centered-accuracy baselines released by the DCLM project. Table 11 lists the full task–category mapping.

DCLM Core (centered accuracy). For each task t with raw accuracy a_t and DCLM-v2 reference baseline ρ_t (the “Random baseline” field in DCLM’s evaluation metadata), the centered score is

$$c_t = \frac{a_t - \rho_t}{1 - \rho_t}, \quad (11)$$

so that 0 corresponds to the DCLM-v2 reference baseline and 1 to perfect accuracy. DCLM Core is the unweighted mean of c_t over the 22 tasks.

DCLM Completion (gold-completion micro loss). Nine of the 22 tasks ask for a free-form answer rather than a choice among options, so every item in them has a gold answer string: CoQA, SQuAD, Jeopardy,

LAMBADA, BB QA Wikidata, BB CS Algorithms, BB Dyck Languages, BB Operators, and BB Repeat Copy. For each item we compute the per-token cross-entropy loss on the ground-truth answer tokens (the “gold completion”). DCLM Completion is the token-weighted (micro) average of these losses over all items of those nine tasks. Because it is a loss rather than an accuracy, lower is better. This metric complements Core by providing a continuous, fine-grained signal that is less affected by the discretization inherent in accuracy.

Multi-seed protocol. Each few-shot task is evaluated with 10 independent random seeds that determine the selection and ordering of the few-shot demonstrations. Reported scores are the mean over these 10 runs. Zero-shot tasks (e.g. LAMBADA, Winograd) are deterministic and therefore evaluated with a single run. All evaluations preserve the beginning-of-sequence (BOS) token when truncating context to the model’s maximum sequence length.

Table 11 Evaluation tasks by domain category, each cited to its original source. The 22 DCLM Core tasks are grouped into five categories following the official DCLM; MMLU 5-shot is listed separately from the Core set. Parentheses give the number of tasks in each category, and the right column the number of in-context demonstrations used.

Category	Tasks	Shots
Reading Comprehension (3)	BoolQ [13]	10
	CoQA [52]	0
	SQuAD [51]	10
World Knowledge (4)	ARC-Challenge [14]	10
	ARC-Easy [14]	10
	BB QA Wikidata [59]	10
	Jeopardy [31]	10
Commonsense Reasoning (4)	CommonsenseQA [61]	10
	COPA [53]	0
	OpenBookQA [46]	0
	PIQA [8]	10
Language Understanding (6)	BB Language ID [59]	10
	HellaSwag 10-shot [67]	10
	HellaSwag 0-shot [67]	0
	LAMBADA [47]	0
	Winograd [39]	0
	Winogrande [55]	0
Symbolic Problem Solving (5)	LSAT AR [69]	3
	BB CS Algorithms [59]	10
	BB Dyck Languages [59]	10
	BB Operators [59]	10
	BB Repeat Copy [59]	10
<i>Reported separately</i> (1)	MMLU 5-shot [25]	5